BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC NHA TRANG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC

TÊN ĐỀ TÀI:

XÂY DỰNG THƯ VIỆN NHẬN DẠNG THẾ CỜ TƯỚNG TỪ ẢNH BẰNG YOLO11 VÀ GỢI Ý NƯỚC ĐI

Giảng viên hướng dẫn:	GV.Nguyễn Hải Triều
Nhóm sinh viên thực hiện:	**Nhóm 2**
Thành viên:	Nguyễn Hữu Trọng – 65133958
	Nguyễn Gia Khiêm – 65131478
	Huỳnh Thanh Trình – 65133949

Nha Trang, tháng 11 năm 2026

THÀNH VIÊN VÀ PHÂN CÔNG NHIỆM VỤ

Đề tài được thực hiện bởi Nhóm 2 gồm ba sinh viên. Sinh viên Nguyễn Hữu Trọng là nhóm trưởng, chịu trách nhiệm chính về định hướng kỹ thuật, mô hình nhận dạng, thuật toán và thư viện; các thành viên còn lại phụ trách dữ liệu, tiền xử lý ảnh, robot JetMax, giao diện ứng dụng và kiểm thử. Mỗi phần việc đều có một thành viên khác kiểm tra chéo trước khi tổng hợp.

Bảng thành viên Nhóm 2

STT	Họ và tên	MSSV	Vai trò
1	Nguyễn Hữu Trọng	65133958	Nhóm trưởng – phụ trách chính
2	Nguyễn Gia Khiêm	65131478	Thành viên
3	Huỳnh Thanh Trình	65133949	Thành viên

Bảng phân công nhiệm vụ

STT	Nội dung công việc	Người thực hiện	Người kiểm tra
1	Khảo sát đề tài, lập kế hoạch, thiết kế kiến trúc thư viện (Chương 1, 3)	Nguyễn Hữu Trọng	Trình
2	Tìm hiểu cơ sở lý thuyết xử lý ảnh, YOLO11, thư viện Ultralytics (Chương 2)	Nguyễn Gia Khiêm Nguyễn Hữu Trọng	Khiêm, Trọng
3	Bước 1 - Thu thập 4 bộ dữ liệu, chuẩn hoá nhãn, khử ảnh trùng (pHash), chia tập	Nguyễn Gia Khiêm	Trọng
4	Tiền xử lý ảnh: letterbox, HSV, phân ngưỡng, hình thái học, Hough, homography; khảo sát CLAHE, gamma, cân bằng trắng, lọc song phương (Chương 4)	Nguyễn Hữu Trọng Nguyễn Gia Khiêm	Trình
5	Bước 2 - Huấn luyện hai giai đoạn YOLO11n, đánh giá, kiểm tra xoay, xuất ONNX (Chương 5)	Nguyễn Hữu Trọng	Khiêm
6	Bước 3 - Tìm hiểu robot JetMax, Jetson Nano; biên dịch Pikafish cho ARM	Huỳnh Thanh Trình	Trọng

STT	Nội dung công việc	Người thực hiện	Người kiểm tra
7	Bước 4 - Thuật toán khớp lưới 9 × 10, đánh giá ảnh → FEN (Chương 6)	Nguyễn Hữu Trọng	Khiêm
8	Bước 5 - Luật cờ, perft, kiểm tra thế cờ, suy ra nước đi và tự sửa lỗi, tích hợp Pikafish	Nguyễn Hữu Trọng	Trình
9	Bước 6 - Thư viện XiangqiBrain, tài liệu API, kiểm thử tự động, đóng gói cho robot	Nguyễn Hữu Trọng Huỳnh Thanh Trình	Khiêm
10	Giao diện ứng dụng Gradio, gói demo Windows, chụp ảnh thử nghiệm (Chương 7)	Huỳnh Thanh Trình	Trọng
11	Tổng hợp, viết và định dạng báo cáo; chuẩn bị slide thuyết trình	Nguyễn Hữu Trọng Nguyễn Gia Khiêm Huỳnh Thanh Trình	Cả nhóm

Bảng mức độ đóng góp của các thành viên

STT	Họ và tên	Phần việc chính	Mức đóng góp
1	Nguyễn Hữu Trọng	Mô hình, thuật toán, thư viện, báo cáo	50%
2	Nguyễn Gia Khiêm	Dữ liệu, tiền xử lý ảnh, lý thuyết	25%
3	Huỳnh Thanh Trình	Robot JetMax, giao diện, kiểm thử	25%

MỤC LỤC

THÀNH VIÊN VÀ PHÂN CÔNG NHIỆM VỤ i

MỤC LỤC iii

DANH SÁCH BẢNG BIỂU vii

DANH SÁCH HÌNH ẢNH ix

DANH MỤC TỪ VIẾT TẮT xiii

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI 1

1.1. Lý do chọn đề tài 1

1.2. Mục tiêu nghiên cứu 1

1.3. Đối tượng và phạm vi nghiên cứu 2

1.4. Phương pháp nghiên cứu 2

1.5. Đóng góp chính của đề tài 3

1.6. Cấu trúc báo cáo 3

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG 4

2.1. Bài toán cờ tướng 4

2.1.1. Bàn cờ và quân cờ 4

2.1.2. Biểu diễn thế cờ: FEN, UCI và ký hiệu tiếng Việt 4

2.2. Cơ sở xử lý ảnh số 5

2.2.1. Ảnh số và không gian màu 5

2.2.2. Phân ngưỡng (thresholding) 6

2.2.3. Phép toán hình thái học 6

2.2.4. Phát hiện biên và biến đổi Hough 6

2.2.5. Biến đổi hình học và homography 6

2.2.6. Chuẩn hoá kích thước, chuẩn hoá giá trị và tăng cường ảnh 7

2.3. Học sâu và bài toán phát hiện đối tượng 7

2.3.1. Mạng nơ-ron tích chập 7

2.3.2. Họ mô hình YOLO 7
2.3.3. Kiến trúc YOLO11 8
2.3.4. Hàm mất mát 8
2.3.5. Các chỉ số đánh giá 9
2.4. Thư viện Ultralytics 9
2.4.1. Giới thiệu 9
2.4.2. Các chế độ sử dụng 10
2.4.3. Định dạng dữ liệu 10
2.4.4. Những gì thư viện tự động làm và những gì đề tài tự xây dựng 11
2.4.5. Tham số huấn luyện chính 11
2.5. Triển khai mô hình: ONNX, ONNX Runtime và TensorRT 12
2.6. Engine cờ tướng và giao thức UCI 12
2.7. Công cụ và môi trường phát triển 13
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 14
3.1. Phân tích yêu cầu 14
3.1.1. Yêu cầu chức năng 14
3.1.2. Yêu cầu phi chức năng 14
3.2. Kiến trúc tổng thể 15
3.3. Thiết kế thư viện 16
3.4. Môi trường triển khai 17
CHƯƠNG 4. XÂY DỰNG DỮ LIỆU VÀ TIỀN XỬ LÝ ẢNH 19
4.1. Nguồn dữ liệu 19
4.2. Làm sạch và chuẩn hoá nhãn 20
4.3. Phát hiện ảnh trùng bằng băm cảm nhận 22
4.4. Chia tập huấn luyện, kiểm định và kiểm tra 23
4.5. Tiền xử lý ảnh đầu vào mô hình 24

4.5.1. Lựa chọn kích thước ảnh đầu vào 25
4.6. Tăng cường dữ liệu 26
4.7. Định vị bàn cờ bằng xử lý ảnh cổ điển 28
4.7.1. Bước 1 - Chuyển sang không gian HSV 28
4.7.2. Bước 2 - Phân ngưỡng màu đỏ 28
4.7.3. Bước 3 - Hình thái học và tìm vùng bàn cờ 29
4.7.4. Bước 4 - Biến đổi Hough tìm hai họ đường lưới 29
4.7.5. Bước 5 - Homography và 90 giao điểm 29
4.7.6. Bước 6 - Gán quân vào giao điểm 30
4.8. Xác định màu quân bằng phân ngưỡng HSV 31
4.9. Tăng cường chất lượng ảnh tự động 31
CHƯƠNG 5. HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH NHẬN DẠNG QUÂN CỜ 36
5.1. Lựa chọn mô hình 36
5.2. Phương pháp huấn luyện hai giai đoạn 36
5.3. Quá trình huấn luyện 37
5.4. Kết quả trên tập kiểm tra 39
5.5. Độ bền khi quân cờ bị xoay 42
5.6. Ví dụ dự đoán 43
5.7. Xuất mô hình sang ONNX và kiểm tra 44
CHƯƠNG 6. TỪ ẢNH ĐẾN THẾ CỜ VÀ NƯỚC ĐI 46
6.1. Khớp lưới 9 × 10 từ tâm quân 46
6.2. Đánh giá chuỗi ảnh → FEN trên tập kiểm tra 47
6.2.1. Tạo FEN thật cho ảnh kiểm tra 47
6.2.2. Kết quả 48
6.3. Luật cờ và kiểm chứng bộ sinh nước đi 50

6.4. Kiểm tra thế cờ nhận dạng được 51
6.5. Suy ra nước vừa đi và tự sửa lỗi nhận dạng 52
6.5.1. Thực nghiệm mô phỏng 53
6.5.2. Thực nghiệm với lỗi nhận dạng thật 54
6.6. Chọn ngưỡng tin cậy 55
6.7. Engine Pikafish 55
CHƯƠNG 7. XÂY DỰNG THƯ VIỆN, GIAO DIỆN ỨNG DỤNG VÀ TRIỂN KHAI 57
7.1. Thư viện XiangqiBrain 57
7.2. Kiểm thử tự động 57
7.3. Giao diện ứng dụng 59
7.4. Đóng gói và triển khai lên robot JetMax 61
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 63
Kết quả đạt được 63
Hạn chế 63
Hướng phát triển 64
TÀI LIỆU THAM KHẢO 65
PHỤ LỤC 67
Phụ lục A. Cấu trúc mã nguồn 67
Phụ lục B. Hướng dẫn chạy giao diện ứng dụng trên Windows 67

DANH SÁCH BẢNG BIỂU

Bảng 2. 1. Luật đi của các quân và điều kiện bị cản 4

Bảng 2. 2. Các chế độ (mode) của thư viện Ultralytics được dùng trong đề tài 10

Bảng 2. 3. Phân chia công việc giữa thư viện Ultralytics và mã nguồn của đề tài 11

Bảng 2. 4. Ý nghĩa các tham số huấn luyện của Ultralytics được sử dụng 11

Bảng 2. 5. Công cụ, thư viện và môi trường sử dụng.....13

Bảng 3. 1. Yêu cầu chức năng của thư viện 14

Bảng 3. 2. Bốn tầng xử lý của hệ thống 15

Bảng 3. 3. Thông số chính của robot JetMax và bộ điều khiển Jetson Nano.....17

Bảng 4. 1. Bốn bộ dữ liệu nguồn (số ảnh thô theo tập gốc) 19

Bảng 4. 2. Quy tắc làm sạch và chuẩn hoá nhãn 20

Bảng 4. 3. Kết quả chia tập theo nguồn (số ảnh).....23

Bảng 4. 4. Các phép tăng cường dữ liệu và lý do lựa chọn.....26

Bảng 4. 5. Các kỹ thuật tăng cường chất lượng ảnh trong mô-đun vision/enhance.py.32

Bảng 4. 6. Chế độ tự động trên ảnh suy giảm mô phỏng: các bước được bật và kết quả định vị bàn cờ trước / sau tăng cường 34

Bảng 5. 1. Tham số huấn luyện của hai giai đoạn (ô in đậm là thay đổi) 37

Bảng 5. 2. Kết quả trên tập val tại epoch tốt nhất (epoch 60) 38

Bảng 5. 3. Kết quả tổng hợp trên tập kiểm tra 39

Bảng 5. 4. Kết quả theo từng lớp trên tập kiểm tra 39

Bảng 5. 5. mAP@0.5 trên tập kiểm tra khi xoay toàn bộ ảnh.....42

Bảng 6. 1. Tỉ lệ khôi phục đúng toàn bộ thế cờ trên dữ liệu tổng hợp 47

Bảng 6. 2. Các bước lọc tạo tập đánh giá ảnh → FEN.....47

Bảng 6. 3. Kết quả ảnh → FEN trên 229 ảnh đánh giá (ngưỡng tin cậy 0,5) 48

Bảng 6. 4. Kiểm chứng bộ sinh nước đi bằng perft từ thế khai cuộc 51

Bảng 6. 5. Ví dụ phát hiện thế cờ vô lý 52

Bảng 6. 6. Hiệu quả sửa lỗi bằng luật cờ (mô phỏng) 53
Bảng 6. 7. Tin thẳng ảnh và suy ra nước đi bằng luật cờ trên lỗi nhận dạng thật 54
Bảng 6. 8. Ảnh hưởng của ngưỡng tin cậy 55
Bảng 7. 1. Các hàm chính của lớp XiangqiBrain 57

DANH SÁCH HÌNH ẢNH

Hình 2. 1. Thế khai cuộc theo quy ước toạ độ của đề tài (Đỏ ở hàng 0).....5

Hình 2. 2. Sơ đồ kiến trúc YOLO11 (vẽ lại theo cấu hình yolo11.yaml của Ultralytics) 8

Hình 2. 3. Minh hoạ chỉ số IoU giữa hộp thật và hộp dự đoán 9

Hình 3. 1. Vòng lặp hoạt động của robot đánh cờ và các giai đoạn thực hiện đề tài 15

Hình 3. 2. Sơ đồ các mô-đun của thư viện XiangqiBrain 16

Hình 3. 3. Luồng xử lý của hàm XiangqiBrain.analyze() 16

Hình 3. 4. Robot Hiwonder JetMax: tổng thể, cấu tạo và kích thước (nguồn: Hiwonder / OpenELAB).....17

Hình 3. 5. Sơ đồ khối phần cứng hệ thống robot JetMax.....18

Hình 3. 6. Mô phỏng tầm với của JetMax so với bàn cờ 360 × 260 mm (nhìn từ trên)18

Hình 4. 1. Hai nhánh tiền xử lý ảnh của hệ thống: nhánh nhận diện quân và nhánh định vị bàn cờ 19

Hình 4. 2. Quy trình xây dựng bộ dữ liệu hợp nhất merged_v1 20

Hình 4. 3. Số ảnh thô của từng bộ theo tập gốc.....20

Hình 4. 4. Chuyển nhãn polygon (trái) và nhãn pose (phải) về hộp bao.....21

Hình 4. 5. Số quân theo lớp (xếp chồng theo nguồn) và số quân mỗi ảnh của từng bộ22

Hình 4. 6. Phân bố kích thước nhóm ảnh trùng và mức rò rỉ nếu dùng cách chia gốc .22

Hình 4. 7. Một nhóm ảnh trùng (44 ảnh) – phải nằm cùng một tập.....23

Hình 4. 8. Số ảnh mỗi tập và phân bố lớp giữa ba tập (càng giống nhau càng tốt) 23

Hình 4. 9. Mẫu của 14 lớp quân cần nhận diện (hàng trên: quân Đen, hàng dưới: quân Đỏ).....24

Hình 4. 10. Kiểm tra nhãn bằng mắt: ảnh ngẫu nhiên từ các bộ sau khi hợp nhất.....24

Hình 4. 11. Tiền xử lý ảnh đầu vào: ảnh gốc, letterbox 640 × 640, tensor một kênh và phân bố giá trị sau chuẩn hoá 25

Hình 4. 12. Kích thước quân cờ sau khi đưa về 640 px và mật độ quân trong mỗi ảnh 26

Hình 4. 13. Minh hoạ tăng cường: xoay và đổi màu được dùng; lật ảnh bị loại vì tạo chữ ngược gương.....27

Hình 4. 14. Thống kê nhãn và các lô ảnh huấn luyện sau tăng cường (mosaic, xoay, đổi màu) do Ultralytics sinh ra 27

Hình 4. 15. Ảnh bàn cờ thật và ba kênh H, S, V 28

Hình 4. 16. Histogram kênh H và mặt nạ đỏ tạo từ hai khoảng sắc độ.....28

Hình 4. 17. Mặt nạ đỏ trước và sau phép đóng, các điểm ảnh được lấp và đường viền vùng bàn cờ.....29

Hình 4. 18. Biên Canny (tham khảo), các đoạn thẳng Hough tách thành hai họ đường và histogram góc 29

Hình 4. 19. 90 giao điểm tìm được trên ảnh thật và ảnh bàn cờ sau khi nắn phối cảnh30

Hình 4. 20. Vùng chấp nhận bán kính 0,45 ô quanh mỗi giao điểm.....30

Hình 4. 21. Xác định màu quân: ảnh quân, điểm "mực đỏ" và điểm "mực tối" tìm được bằng phân ngưỡng HSV 31

Hình 4. 22. Ảnh bị suy giảm (thiếu sáng, ám màu, nhiễu) và kết quả sau các kỹ thuật tăng cường 33

Hình 5. 1. So sánh độ chính xác – chi phí tính toán của các mô hình phát hiện đối tượng (COCO val2017).....36

Hình 5. 2. Quá trình huấn luyện hai giai đoạn và triển khai mô hình 36

Hình 5. 3. Theo dõi quá trình tinh chỉnh: box/cls/dfl loss, Precision – Recall, mAP và learning rate trên tập val 38

Hình 5. 4. mAP trên tập val qua hai giai đoạn huấn luyện.....38

Hình 5. 5. mAP theo từng lớp và quan hệ Precision – Recall của từng lớp trên tập kiểm tra 40

Hình 5. 6. Ma trận nhầm lẫn trên tập kiểm tra (chuẩn hoá theo nhãn thật).....41

Hình 5. 7. Đường cong Precision – Recall và F1 – Confidence của 14 lớp.....41

Hình 5. 8. Độ chính xác theo góc xoay của hai mô hình.....42

Hình 5. 9. Cùng một ảnh kiểm tra xoay 4 góc: mô hình cũ bỏ sót / nhầm nhiều ở 90° và 270°, mô hình mới ổn định.....42

Hình 5. 10. So sánh nhãn thật (trái) và dự đoán của mô hình (phải) trên một ảnh: 16/16 quân đúng (TP = 16, FP = 0, FN = 0).....43

Hình 5. 11. Dự đoán trên ảnh bàn cờ chụp nghiêng, có lóa sáng.....43

Hình 5. 12. Dự đoán trên ảnh bàn cờ thật nằm nghiêng (bộ scara).....44

Hình 5. 13. Hành trình của mô hình từ Colab tới robot 44

Hình 5. 14. Các phép tính trong best.onnx, so sánh đầu ra PyTorch – ONNX và tốc độ trên CPU 45

Hình 5. 15. Kết quả PyTorch (trái) và ONNX + NMS tự viết (phải) trên cùng một ảnh: đều 32 quân.....45

Hình 6. 1. Luồng xử lý ảnh → FEN bằng khớp lưới từ tâm quân 46

Hình 6. 2. Lưới khớp được (chấm xanh) trên ảnh kiểm tra và FEN thật tương ứng.....48

Hình 6. 3. Phân bố số ô sai mỗi ảnh (xanh: đúng hoàn toàn, cam: luật cờ còn sửa được, đỏ: quá nhiều lỗi).....49

Hình 6. 4. Ví dụ ảnh tốt nhất, trung vị và tệ nhất: ảnh với lưới khớp được, FEN thật và FEN dự đoán (ô sai tô đỏ) 50

Hình 6. 5. Nước đi hợp lệ của từng loại quân trong các thế cờ có quân cản (chấm xanh: đi được, ×: ăn được, vòng hồng: quân cản).....51

Hình 6. 6. Ví dụ: ảnh mới có 2 ô nhận sai (pháo b7 nhận thành xe, sót tốt i6) – thư viện vẫn suy ra đúng nước "Pháo 2 bình 5" và sửa cả 2 ô 53

Hình 6. 7. Hiệu quả sửa lỗi nhận diện bằng luật cờ theo tỉ lệ lỗi (mô phỏng) 54

Hình 6. 8. Ảnh thật, lỗi nhận diện thật: tin thẳng ảnh và suy ra nước đi bằng luật cờ..54

Hình 6. 9. Tỉ lệ đúng theo ngưỡng tin cậy; ngưỡng 0,25 được chọn làm mặc định của thư viện 55

Hình 6. 10. Thời gian tìm nước của Pikafish theo độ sâu trên 3 thế cờ (CPU Colab)..56

Hình 7. 1. Thư viện chạy trọn chuỗi ảnh → nước đi trên ảnh kiểm tra (trên) và phát hiện một thế cờ vô lý (dưới) 58

Hình 7. 2. Giao diện thẻ “Phân tích một ảnh”: ảnh tải lên, quân nhận diện được, thế cờ và nước gợi ý 60

Hình 7. 3. Giao diện thẻ “Chơi cả ván”: máy trả lời nước đi và ghi biên bản ván cờ... 61

Hình 7. 4. Quy trình đóng gói trên Colab và cài đặt trên robot JetMax 62

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Ý nghĩa
AP / mAP	Average Precision / mean Average Precision	Độ chính xác trung bình (theo lớp / trung bình các lớp)
BGR / RGB / HSV / LAB	Color spaces	Các không gian màu
CLAHE	Contrast Limited Adaptive Histogram Equalization	Cân bằng histogram thích nghi có giới hạn tương phản
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
DFL	Distribution Focal Loss	Hàm mất mát phân phối cho toạ độ hộp
FEN	Forsyth–Edwards Notation	Chuỗi ký hiệu mô tả một thế cờ
FP16 / FP32	Floating point 16/32-bit	Số thực 16/32 bit
GFLOPs	Giga Floating-point Operations	Tỉ phép tính dấu phẩy động
IoU	Intersection over Union	Tỉ lệ giao / hợp giữa hai hộp
NMS	Non-Maximum Suppression	Loại bỏ hộp trùng, giữ hộp điểm cao nhất
NNUE	Efficiently Updatable Neural Network	Mạng nơ-ron đánh giá thế cờ của Pikafish
ONNX	Open Neural Network Exchange	Định dạng mô hình mở, trung gian

Viết tắt	Tiếng Anh	Ý nghĩa
P / R	Precision / Recall	Độ chính xác / Độ bao phủ
pHash	Perceptual Hash	Băm cảm nhận, dùng tìm ảnh gần trùng
UCI	Universal Chess Interface	Giao thức giao tiếp với engine cờ
YOLO	You Only Look Once	Họ mô hình phát hiện đối tượng một giai đoạn

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1. Lý do chọn đề tài

Cờ tướng là môn cờ trí tuệ phổ biến bậc nhất tại Việt Nam và các nước châu Á. Các phần mềm chơi cờ hiện nay đều đòi hỏi người chơi thao tác trên màn hình; trong khi đó, phần lớn người chơi vẫn quen với bàn cờ và quân cờ thật. Để máy tính (hoặc một cánh tay robot) có thể "chơi cờ thật" cùng con người, hệ thống phải giải quyết được bài toán cốt lõi: **nhìn ảnh bàn cờ, hiểu thế cờ hiện tại và đưa ra nước đi tiếp theo**.

Đây là bài toán kết hợp nhiều kỹ thuật của học phần Xử lý ảnh: thu nhận và tiền xử lý ảnh, biến đổi không gian màu, phân ngưỡng, hình thái học, phát hiện đường thẳng, biến đổi hình học (homography), cùng với nhận dạng đối tượng bằng học sâu. Quân cờ tướng có đặc thù riêng: 14 loại quân, chỉ khác nhau ở **chữ Hán** khắc trên mặt quân và màu mực (đỏ / đen), quân có thể nằm xoay theo mọi hướng, ánh sáng và kiểu bàn cờ rất đa dạng. Vì vậy đề tài vừa có tính thực tiễn, vừa là môi trường tốt để vận dụng và đánh giá các kỹ thuật xử lý ảnh.

Bên cạnh đó, bài toán còn có một đặc điểm thú vị: kết quả nhận dạng **phải tuân theo luật cờ**. Điều này cho phép dùng tri thức về luật cờ để kiểm tra và **tự sửa lỗi nhận dạng** - một hướng tiếp cận ít gặp trong các bài toán nhận dạng đối tượng thông thường.

1.2. Mục tiêu nghiên cứu

Mục tiêu tổng quát: xây dựng một **thư viện phần mềm** nhận ảnh bàn cờ tướng (kèm thế cờ ở lượt trước) và trả về **nước đi tiếp theo** dưới dạng ô gắp - ô đặt, có thể nhúng vào chương trình điều khiển robot JetMax hoặc chạy độc lập trên máy tính qua giao diện ứng dụng.

Các mục tiêu cụ thể:

- Xây dựng bộ dữ liệu hợp nhất cho 14 lớp quân cờ từ nhiều nguồn công khai, làm sạch nhãn, loại bỏ rò rỉ dữ liệu giữa các tập.
- Thiết kế và trình bày rõ **các bước tiền xử lý ảnh** (preprocessing): chuẩn hoá kích thước, chuyển đổi không gian màu, tăng cường dữ liệu, định vị bàn cờ bằng các kỹ thuật xử lý ảnh cổ điển.

- Huấn luyện mô hình phát hiện quân cờ **YOLO11n** bằng thư viện **Ultralytics**, đạt mAP@0.5 ≥ 0.95 trên tập kiểm tra và không suy giảm khi quân bị xoay.
- Chuyển ảnh thành thế cờ (chuỗi FEN), dùng luật cờ để kiểm tra, suy ra nước đối thủ vừa đi và tự sửa lỗi nhận dạng.
- Tích hợp engine cờ tướng Pikafish để tìm nước đi tiếp theo.
- Đóng gói thành thư viện có tài liệu API, xây dựng **giao diện ứng dụng** demo và gói cài đặt cho robot JetMax (Jetson Nano).

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: ảnh chụp bàn cờ tướng và 32 quân cờ thuộc 14 lớp (7 loại quân × 2 màu); các kỹ thuật xử lý ảnh và mô hình học sâu phát hiện đối tượng; luật cờ tướng và engine cờ.

Phạm vi nghiên cứu:

- Ảnh đầu vào là ảnh màu chụp bàn cờ từ phía trên (thẳng hoặc hơi nghiêng), từ camera robot, điện thoại hoặc ảnh có sẵn.
- Mô hình nhận dạng được huấn luyện và đánh giá trên dữ liệu công khai (Roboflow Universe).
- Đầu ra của thư viện là toạ độ ô trên bàn cờ (ví dụ b9 → c7).
- Chưa xử lý các luật nâng cao như cấm lặp nước, đuổi dai (engine Pikafish tự xử lý khi nhận đủ lịch sử ván).

1.4. Phương pháp nghiên cứu

- **Nghiên cứu lý thuyết:** tìm hiểu xử lý ảnh số (không gian màu, phân ngưỡng, hình thái học, biến đổi Hough, homography), mạng nơ-ron tích chập, họ mô hình YOLO, thư viện Ultralytics, giao thức UCI và engine cờ.
- **Thực nghiệm:** xây dựng dữ liệu, huấn luyện, đánh giá định lượng bằng các chỉ số chuẩn (Precision, Recall, mAP), đánh giá trọn chuỗi ảnh → FEN và mô phỏng lỗi nhận dạng.
- **Kiểm chứng chéo:** đối chiếu bộ sinh nước đi tự viết với giá trị perft chuẩn và với engine Pikafish; đối chiếu đầu ra mô hình PyTorch và ONNX.

- **Phát triển phần mềm:** viết thư viện Python tương thích Python 3.6 (robot) đến 3.13 (Colab), kiểm thử tự động, đóng gói và xây dựng giao diện.

1.5. Đóng góp chính của đề tài

1) Bộ dữ liệu hợp nhất 6 808 ảnh, 112 938 quân cờ, 14 lớp, chia tập theo **nhóm ảnh trùng** để không rò rỉ dữ liệu.
2) Mô hình YOLO11n (2,6 triệu tham số) đạt **mAP@0.5 = 0.964** trên 713 ảnh kiểm tra và giữ nguyên độ chính xác khi ảnh xoay 90°/180°/270°.
3) Thuật toán **khớp lưới 9 × 10 từ tâm quân** (không cần biết kiểu bàn cờ) và quy trình định vị bàn cờ bằng xử lý ảnh cổ điển.
4) Cơ chế **dùng luật cờ để tự sửa lỗi nhận dạng**: nếu chỉ dùng ảnh, thế cờ đúng hoàn toàn ở 78,2% số ảnh; khi kết hợp thế cờ lượt trước, hệ thống suy ra đúng nước đi ở 95,2% số ảnh, phần còn lại tự báo "chụp lại" và **không có trường hợp nào sai mà không phát hiện**.
5) Thư viện XiangqiBrain có tài liệu API, bộ kiểm thử tự động, giao diện ứng dụng web và gói cài đặt chạy offline trên robot JetMax.

1.6. Cấu trúc báo cáo

Ngoài phần mở đầu và kết luận, báo cáo gồm 7 chương:

- **Chương 1** - Tổng quan về đề tài.
- **Chương 2** - Cơ sở lý thuyết và công nghệ: cờ tướng, xử lý ảnh số, học sâu và YOLO, thư viện Ultralytics, ONNX/TensorRT, engine cờ.
- **Chương 3** - Phân tích và thiết kế hệ thống.
- **Chương 4** - Xây dựng dữ liệu và tiền xử lý ảnh (trọng tâm của học phần).
- **Chương 5** - Huấn luyện và đánh giá mô hình nhận dạng quân cờ.
- **Chương 6** - Từ ảnh đến thế cờ và nước đi: khớp lưới, luật cờ, tự sửa lỗi, engine.
- **Chương 7** - Xây dựng thư viện, giao diện ứng dụng và triển khai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Bài toán cờ tướng

2.1.1. Bàn cờ và quân cờ

Bàn cờ tướng gồm 9 đường dọc và 10 đường ngang, tạo thành **90 giao điểm**; quân cờ được đặt trên giao điểm (không đặt trong ô như cờ vua). Giữa bàn là "sông" chia hai nửa; mỗi bên có một "cung" 3 × 3 giao điểm. Mỗi bên có 16 quân thuộc 7 loại: Tướng (1), Sĩ (2), Tượng (2), Mã (2), Xe (2), Pháo (2), Tốt (5). Quân hai bên phân biệt bằng màu mực (đỏ / đen) và một số loại dùng chữ Hán khác nhau (ví dụ 帥 / 將 cho Tướng).

Bảng 2. 1. Luật đi của các quân và điều kiện bị cản

Quân	Cách đi	Giới hạn / bị cản
Tướng (K)	1 ô ngang hoặc dọc	Chỉ trong cung; hai tướng không được đối mặt trên cùng cột khi không có quân chắn
Sĩ (A)	1 ô chéo	Chỉ trong cung (5 điểm)
Tượng (B)	2 ô chéo	Không qua sông; bị cản nếu ô giữa ("mắt tượng") có quân
Mã (N)	1 thẳng + 1 chéo (chữ L)	Bị cản nếu ô liền kề theo hướng đi thẳng ("chân mã") có quân
Xe (R)	Thẳng, không giới hạn	Dừng khi gặp quân
Pháo (C)	Đi như xe khi không ăn	Ăn quân phải nhảy qua đúng 1 quân ("ngòi")
Tốt (P)	Tiến 1 ô	Qua sông mới được đi ngang; không bao giờ lùi

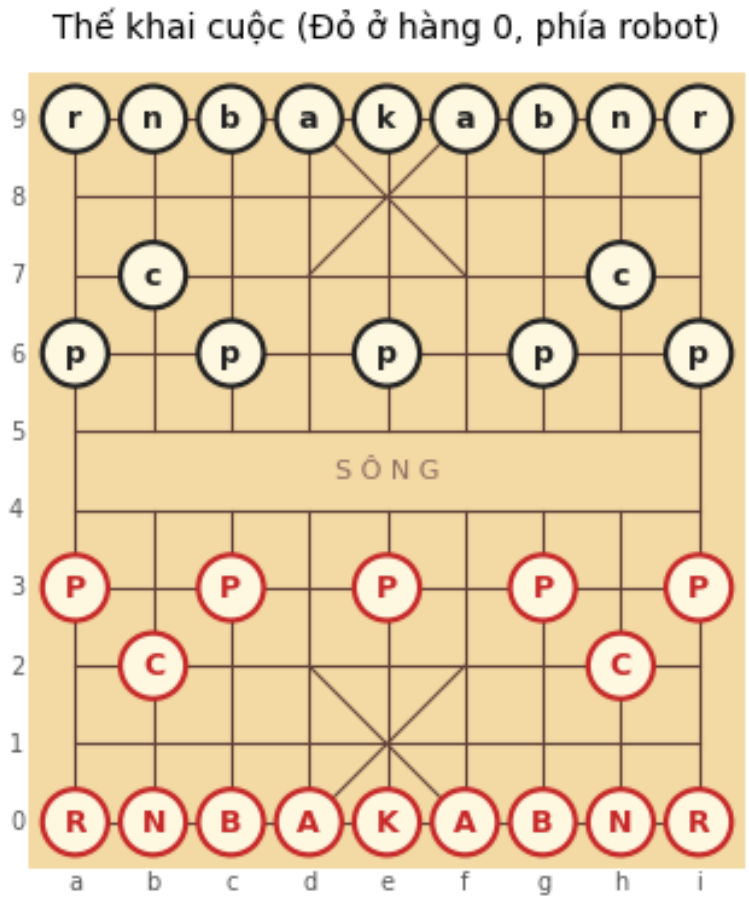
2.1.2. Biểu diễn thế cờ: FEN, UCI và ký hiệu tiếng Việt

Đề tài dùng hệ toạ độ (cột, hàng): cột a–i (trái → phải nhìn từ phía quân Đỏ), hàng 0–9 (quân Đỏ ở hàng 0–4). Một thế cờ được mã hoá bằng chuỗi **FEN** liệt kê từ hàng 9 xuống hàng 0, chữ HOA là quân Đỏ, chữ thường là quân Đen, chữ số là số ô trống liên tiếp; ký tự cuối chỉ bên đi (w = Đỏ, b = Đen). Ví dụ thế khai cuộc:

```
rnbakabnr/9/1c5c1/p1p1p1p1p/9/9/P1P1P1P1P/1C5C1/9/RNBAKABNR w - - 0 1
```

Nước đi dùng ký hiệu **UCI** gồm ô đi và ô đến, ví dụ h2e2. Để hiển thị cho người chơi Việt Nam, thư viện đổi sang ký hiệu truyền thống: cột đánh số 1 - 9 từ phải sang

trái theo góc nhìn của bên đi, kèm hành động *tiến / thoái / bình*; ví dụ h2e2 = “Pháo 2 bình 5”.



Hình 2. 1. Thế khai cuộc theo quy ước toạ độ của đề tài (Đỏ ở hàng 0)

2.2. Cơ sở xử lý ảnh số

2.2.1. Ảnh số và không gian màu

Ảnh màu số là ma trận ba chiều H × W × 3, mỗi điểm ảnh gồm ba giá trị 8 bit. OpenCV lưu ảnh theo thứ tự kênh **BGR**, trong khi mô hình học sâu (huấn luyện bằng PyTorch) nhận **RGB**; vì vậy bước đổi thứ tự kênh là bắt buộc trong tiền xử lý.

Không gian **HSV** tách màu thành sắc độ H (Hue), độ bão hoà S (Saturation) và độ sáng V (Value). Ưu điểm của HSV cho bài toán này: màu đỏ của đường kẻ bàn cờ và mực trên quân Đỏ tập trung ở vùng H gần 0° (hai đầu thang H của OpenCV 0 - 179), gần như không phụ thuộc độ sáng - điều mà RGB không có. Không gian **LAB** tách kênh độ sáng L khỏi hai kênh màu a, b, phù hợp để cân bằng sáng mà không làm lệch màu.

$$V = \max(R, G, B); \quad S = (V - \min(R, G, B)) / V; \quad H = \text{góc sắc độ trên vòng màu (OpenCV: 0–179)}$$

2.2.2. Phân ngưỡng (thresholding)

Phân ngưỡng biến ảnh thành ảnh nhị phân: điểm ảnh thuộc vùng quan tâm có giá trị 255, còn lại 0. Với ảnh màu, phân ngưỡng theo khoảng trên từng kênh HSV (hàm cv2.inRange) cho phép tách một màu cụ thể. Vì màu đỏ nằm ở **hai đầu** thang H, mặt nạ đỏ là hợp (OR) của hai khoảng $H \in [0, 10]$ và $H \in [165, 180]$, kèm điều kiện S, V đủ lớn để loại vùng xám/tối.

2.2.3. Phép toán hình thái học

Hình thái học thao tác trên ảnh nhị phân bằng một phần tử cấu trúc (kernel). **Co** (erosion) thu nhỏ vùng trắng, **giãn** (dilation) mở rộng vùng trắng; **đóng** (closing = giãn rồi co) lấp các khe hở nhỏ, nối các đoạn đường bị đứt do quân cờ che; **mở** (opening = co rồi giãn) xoá nhiễu đốm nhỏ. Sau khi đóng, hàm tìm đường viền (cv2.findContours) cho phép xác định vùng bàn cờ lớn nhất trong ảnh.

2.2.4. Phát hiện biên và biến đổi Hough

Biến đổi Hough phát hiện đường thẳng bằng cách biểu diễn mỗi đường dưới dạng tham số (ρ, θ) với $\rho = x \cdot \cos\theta + y \cdot \sin\theta$. Mỗi điểm biên "bỏ phiếu" cho mọi đường đi qua nó; đường có nhiều phiếu là đường thẳng trong ảnh. Biến thể xác suất (cv2.HoughLinesP) trả về trực tiếp các đoạn thẳng (x1, y1, x2, y2), nhanh hơn và phù hợp để dò các đường lưới bàn cờ.

2.2.5. Biến đổi hình học và homography

Khi chụp một mặt phẳng (bàn cờ) từ góc bất kỳ, quan hệ giữa toạ độ trên mặt phẳng (u, v) và toạ độ ảnh (x, y) là một **phép biến đổi phối cảnh (homography)** - ma trận 3 × 3 có 8 bậc tự do:

$$[x \cdot w, y \cdot w, w]^T = H \cdot [u, v, 1]^T, \quad H \in \mathbb{R}^{3x3} \ (h_{33} = 1)$$

Chỉ cần 4 cặp điểm tương ứng (không có 3 điểm thẳng hàng) là xác định được H; khi có nhiều cặp điểm, H được ước lượng bằng bình phương tối thiểu kết hợp RANSAC để loại điểm sai (cv2.findHomography). Biết H, có thể "nắn" ảnh về dạng nhìn thẳng (cv2.warpPerspective) hoặc chiếu 90 giao điểm của bàn cờ lên ảnh.

2.2.6. Chuẩn hoá kích thước, chuẩn hoá giá trị và tăng cường ảnh

Mô hình học sâu yêu cầu đầu vào kích thước cố định. Phép **letterbox** thu ảnh về 640 × 640 nhưng **giữ nguyên tỉ lệ khung hình**, phần thiếu được đệm màu xám (114) - tránh làm méo quân cờ tròn thành elip. Giá trị điểm ảnh được chia cho 255 để đưa về [0, 1].

Các kỹ thuật **tăng cường chất lượng ảnh** cổ điển gồm: hiệu chỉnh gamma ($I_{out} = I_{in}^{\gamma}$), cân bằng histogram và CLAHE (cân bằng histogram thích nghi theo từng ô nhỏ, có giới hạn tương phản để không khuếch đại nhiễu), cân bằng trắng Gray-World (giả thiết trung bình ảnh là màu xám), lọc song phương (bilateral) khử nhiễu nhưng giữ biên.

Tăng cường dữ liệu (data augmentation) là việc tạo thêm biến thể của ảnh huấn luyện (xoay, đổi màu, co giãn, ghép ảnh…) để mô hình học được tính bất biến, giảm quá khớp.

2.3. Học sâu và bài toán phát hiện đối tượng

2.3.1. Mạng nơ-ron tích chập

Mạng nơ-ron tích chập (CNN) học đặc trưng ảnh qua các lớp tích chập: các lớp đầu học biên và kết cấu, các lớp sâu học hình dạng và ngữ nghĩa (ở đây là nét chữ Hán trên mặt quân). Bài toán **phát hiện đối tượng** (object detection) yêu cầu đồng thời xác định **vị trí** (hộp bao - bounding box) và **loại** của nhiều đối tượng trong ảnh; khác với phân loại ảnh chỉ trả về một nhãn cho cả ảnh.

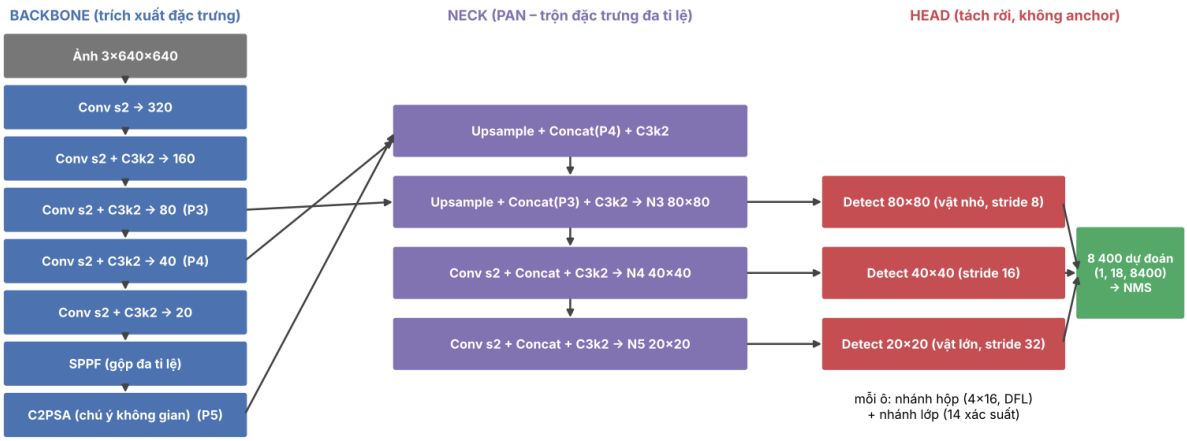
2.3.2. Họ mô hình YOLO

Các bộ phát hiện chia hai nhóm: **hai giai đoạn** (Faster R-CNN: đề xuất vùng rồi phân loại - chính xác nhưng chậm) và **một giai đoạn** (SSD, RetinaNet, YOLO - dự đoán trực tiếp trên lưới đặc trưng, nhanh). YOLO (You Only Look Once) nhìn ảnh một lần duy nhất, phù hợp chạy thời gian thực trên thiết bị nhúng. Các phiên bản gần đây (YOLOv8, YOLO11) bỏ anchor box (anchor-free) và dùng đầu ra tách rời (decoupled head) cho nhánh hộp và nhánh lớp.

2.3.3. Kiến trúc YOLO11

YOLO11 (Ultralytics, 2024) gồm ba phần. **Backbone** trích xuất đặc trưng qua các tầng tích chập bước 2 và khối **C3k2** (biến thể CSP với nhân 3 × 3), kết thúc bằng **SPPF** (gộp đặc trưng ở nhiều kích thước cửa sổ) và **C2PSA** (khối chú ý không gian). **Neck** theo cấu trúc PAN trộn đặc trưng ở ba tỉ lệ 80 × 80, 40 × 40, 20 × 20 (stride 8, 16, 32) để phát hiện vật nhỏ lẫn vật lớn. **Head** dự đoán tại mỗi ô: 4 toạ độ hộp (biểu diễn bằng phân phối rời rạc - DFL) và xác suất của từng lớp.

Với ảnh 640 × 640, số ô dự đoán là $80^2 + 40^2 + 20^2 =$ **8 400**; mỗi ô cho 4 + 14 = 18 giá trị, nên đầu ra của mô hình có kích thước **(1, 18, 8400)**. Bản **YOLO11n** (nano) chỉ có khoảng 2,6 triệu tham số và 6,5 GFLOPs - đủ nhẹ cho Jetson Nano.



Hình 2. 2. Sơ đồ kiến trúc YOLO11 (vẽ lại theo cấu hình yolo11.yaml của Ultralytics)

2.3.4. Hàm mất mát

Tổng mất mát của YOLO11 là tổng có trọng số của ba thành phần (trọng số mặc định box = 7,5; cls = 0,5; dfl = 1,5):

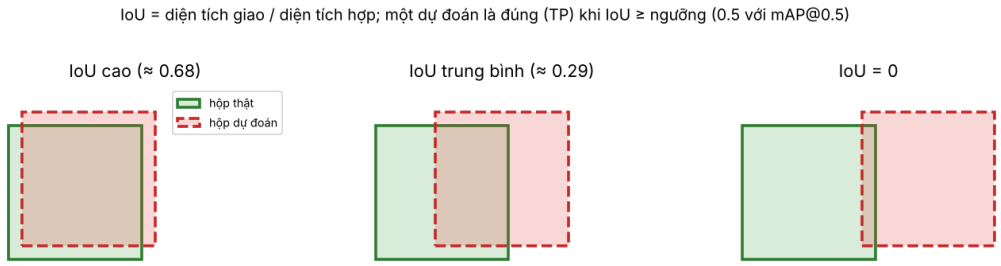
- **Box loss (CIoU):** phạt sai lệch giữa hộp dự đoán và hộp thật dựa trên IoU, khoảng cách tâm và tỉ lệ khung hình.
- **Classification loss (BCE):** entropy chéo nhị phân trên xác suất từng lớp.
- **DFL (Distribution Focal Loss):** học phân phối xác suất của từng cạnh hộp, giúp biên hộp sắc nét.

2.3.5. Các chỉ số đánh giá

IoU (Intersection over Union) đo mức trùng khớp giữa hộp dự đoán A và hộp thật B. Một dự đoán đúng loại và có IoU ≥ ngưỡng được tính là đúng (TP); dự đoán thừa là FP; quân bị bỏ sót là FN.

$$IoU = |A \cap B| / |A \cup B|; \quad Precision = TP / (TP + FP); \quad Recall = TP / (TP + FN)$$

AP (Average Precision) là diện tích dưới đường cong Precision – Recall của một lớp; **mAP@0.5** là trung bình AP của 14 lớp với ngưỡng IoU 0,5; **mAP@0.5:0.95** lấy trung bình thêm trên 10 ngưỡng IoU từ 0,5 đến 0,95 – khắt khe hơn về độ khít của hộp.



Hình 2. 3. Minh hoạ chỉ số IoU giữa hộp thật và hộp dự đoán

2.4. Thư viện Ultralytics

2.4.1. Giới thiệu

Ultralytics là thư viện Python mã nguồn mở (giấy phép AGPL-3.0) do công ty Ultralytics phát triển, cung cấp trọn bộ quy trình cho các mô hình YOLO (YOLOv8, YOLO11, …): huấn luyện, đánh giá, dự đoán và xuất mô hình sang nhiều định dạng. Thư viện xây dựng trên **PyTorch**, cài đặt bằng pip install ultralytics. Đề tài dùng phiên bản 8.4 trên Google Colab (Python 3.13, GPU Tesla T4).

2.4.2. Các chế độ sử dụng

Bảng 2. 2. Các chế độ (mode) của thư viện Ultralytics được dùng trong đề tài

Chế độ	Lệnh Python	Việc thực hiện bên trong	Dùng ở
Train	model.train(data='data.yaml', epochs=60, imgsz=640, ...)	Đọc dữ liệu, tăng cường ảnh, lan truyền xuôi/ngược, cập nhật trọng số, đánh giá trên tập val mỗi epoch, lưu best.pt / last.pt	Chương 5
Val	model.val(data=..., split='test')	Dự đoán trên tập kiểm tra, ghép với nhãn theo IoU, tính P, R, mAP từng lớp, vẽ ma trận nhầm lẫn và đường PR	Chương 5
Predict	model.predict(source, conf=0.25, imgsz=640)	Letterbox → suy luận → giải mã → NMS → đổi toạ độ về ảnh gốc; trả về đối tượng Results	Chương 5, 6
Export	model.export(format='onnx', opset=12, simplify=True)	Chuyển đồ thị PyTorch sang ONNX, rút gọn đồ thị bằng onnxslim	Chương 5

2.4.3. Định dạng dữ liệu

Ultralytics dùng định dạng nhãn YOLO: mỗi ảnh có một tệp .txt, mỗi dòng một đối tượng lớp x_tâm y_tâm rộng cao với toạ độ chuẩn hoá về [0, 1] theo kích thước ảnh. Tệp cấu hình data.yaml khai báo đường dẫn các tập và danh sách tên lớp:

```
path: /content/merged
train: train/images
val: val/images
test: test/images
nc: 14
names: [black_advisor, black_bishop, black_cannon, black_king, black_knight,
black_pawn, black_rook,
        red_advisor, red_bishop, red_cannon, red_king, red_knight, red_pawn,
red_rook]
```

2.4.4. Những gì thư viện tự động làm và những gì đề tài tự xây dựng

Hiểu rõ ranh giới này rất quan trọng khi dùng thư viện, vì trên robot (Python 3.6, không có Internet) không thể cài Ultralytics.

Bảng 2. 3. Phân chia công việc giữa thư viện Ultralytics và mã nguồn của đề tài

Công đoạn	Ultralytics (khi huấn luyện trên Colab)	Đề tài tự cài đặt (chạy trên robot / laptop)
Đọc ảnh, letterbox, chuẩn hoá	Tự động trong DataLoader / predict	vision/pieces.py: hàm letterbox()
Tăng cường dữ liệu	Tự động theo tham số (degrees, hsv_*, mosaic…)	Chọn tham số: tắt lật ảnh, xoay ±180°
Kiến trúc, hàm mất mát, tối ưu	Tự động	—
Giải mã đầu ra (1, 18, 8400) + NMS	Tự động trong predict	vision/pieces.py: hàm decode() + cv2.dnn.NMSBoxes
Chạy mô hình	PyTorch	TensorRT (Jetson) / ONNX Runtime / OpenCV DNN
Đánh giá P, R, mAP	Tự động trong val	Đánh giá ảnh → FEN (Chương 6)

2.4.5. Tham số huấn luyện chính

Bảng 2. 4. Ý nghĩa các tham số huấn luyện của Ultralytics được sử dụng

Tham số	Ý nghĩa
imgsz	Kích thước ảnh đầu vào sau letterbox (640)
epochs, batch, patience	Số lần duyệt dữ liệu; số ảnh mỗi bước; dừng sớm nếu không cải thiện
optimizer, lr0, lrf	Thuật toán tối ưu (SGD / AdamW); learning rate ban đầu; tỉ lệ LR cuối
degrees	Xoay ảnh ngẫu nhiên trong [−degrees, +degrees]
fliplr, flipud	Xác suất lật ngang / lật dọc ảnh
hsv_h, hsv_s, hsv_v	Biên độ thay đổi ngẫu nhiên sắc độ, độ bão hoà, độ sáng

Tham số	Ý nghĩa
mosaic, close_mosaic	Ghép 4 ảnh thành 1; tắt mosaic ở N epoch cuối
scale, translate	Co giãn ±50%, dịch ±10% kích thước ảnh

2.5. Triển khai mô hình: ONNX, ONNX Runtime và TensorRT

ONNX là định dạng mở biểu diễn mô hình dưới dạng đồ thị tính toán, độc lập với framework. Mô hình huấn luyện bằng PyTorch được xuất sang ONNX (opset 12 để tương thích TensorRT 8.0 trên JetPack 4.6), sau đó có thể chạy bằng **ONNX Runtime** (CPU, laptop/Colab) hoặc được **TensorRT** biên dịch thành engine tối ưu cho đúng GPU của Jetson Nano: gộp lớp (Conv + BN + SiLU), chọn kernel nhanh nhất trên chính GPU đó và hạ độ chính xác xuống FP16. Engine TensorRT chỉ chạy được trên đúng máy và đúng phiên bản TensorRT đã build, nên phải build trên robot.

2.6. Engine cờ tướng và giao thức UCI

Engine cờ tìm nước đi bằng **tìm kiếm cây trò chơi**: minimax với cắt tỉa **alpha-beta** loại bỏ các nhánh chắc chắn không tốt hơn nhánh đã biết, kết hợp đào sâu dần và bảng băm. Hàm đánh giá thế cờ của **Pikafish** – engine cờ tướng mã nguồn mở mạnh nhất hiện nay, phát triển từ Stockfish - là mạng nơ-ron **NNUE** cập nhật tăng dần khi đi quân nên rất nhanh trên CPU.

Engine giao tiếp qua giao thức văn bản **UCI**: chương trình gửi position fen <FEN> moves ..., rồi go depth N; engine trả về bestmove h2e2. Nhờ UCI, có thể thay engine khác mà không sửa phần còn lại.

Để kiểm chứng bộ sinh nước đi, đề tài dùng **perft** (performance test): đếm số chuỗi nước đi hợp lệ tới độ sâu d và so với giá trị chuẩn đã công bố. Chỉ cần một luật cài sai (ví dụ quên chân mã) là con số ở độ sâu 3 - 4 sẽ lệch.

2.7. Công cụ và môi trường phát triển

Bảng 2. 5. Công cụ, thư viện và môi trường sử dụng

Thành phần	Phiên bản / mô tả	Vai trò
Python	3.13 (Colab), 3.6.9 (robot), 3.11 (laptop)	Ngôn ngữ chính
Ultralytics + PyTorch	8.4 / 2.11, GPU Tesla T4	Huấn luyện, đánh giá, xuất mô hình
OpenCV, NumPy	4.x	Xử lý ảnh, đại số tuyến tính
ONNX, ONNX Runtime, TensorRT	opset 12 / 1.31 / 8.0.1.6	Chạy mô hình ngoài PyTorch
Roboflow, imagehash, scikit-learn	—	Tải dữ liệu, băm cảm nhận, chia tập theo nhóm
Pikafish	dev-20261029, NNUE 48 MB	Engine cờ tướng
Gradio, Matplotlib	6.x / 3.x	Giao diện ứng dụng, vẽ biểu đồ
Google Colab, Google Drive, GitHub	—	Môi trường chạy, lưu trữ, quản lý mã nguồn
Robot Hiwonder JetMax	Jetson Nano 4 GB, JetPack 4.6.1	Nền tảng triển khai

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu

3.1.1. Yêu cầu chức năng

Bảng 3. 1. Yêu cầu chức năng của thư viện

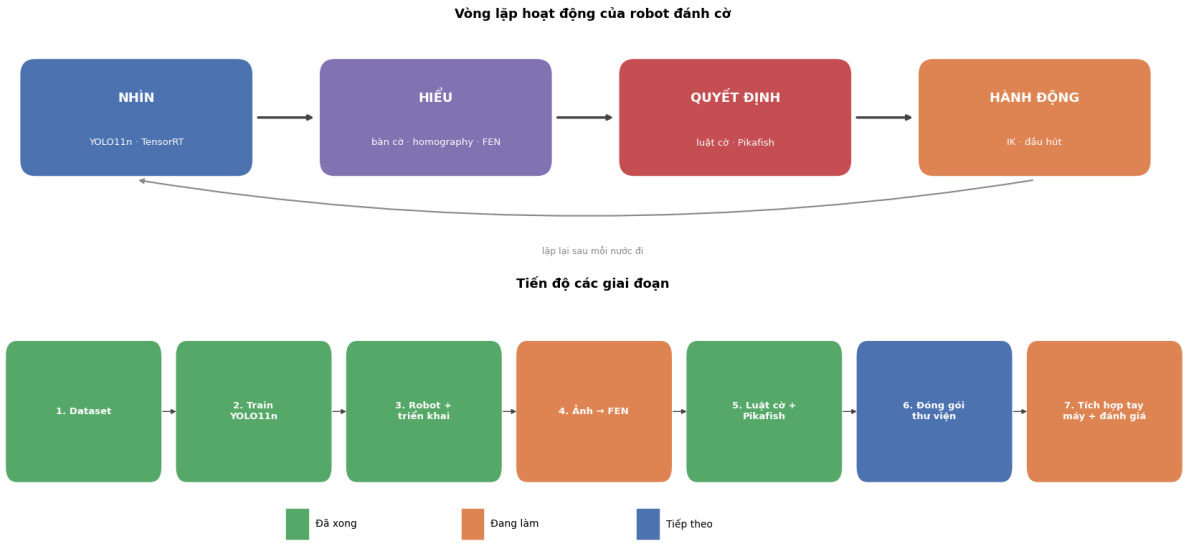
Mã	Chức năng	Mô tả
F1	Nhận diện quân	Phát hiện vị trí và loại của 14 lớp quân trong ảnh, kể cả khi quân xoay bất kỳ
F2	Định vị bàn cờ	Tìm 90 giao điểm của bàn cờ trong ảnh (bàn của đề tài hoặc bàn bất kỳ)
F3	Ảnh → thế cờ	Gán mỗi quân vào giao điểm gần nhất, sinh chuỗi FEN
F4	Kiểm tra thế cờ	Phát hiện thế cờ vô lý (thừa quân, sĩ ra khỏi cung, tượng qua sông…)
F5	Suy ra nước vừa đi	Từ thế cờ lượt trước và ảnh mới, xác định nước đối thủ vừa đi và sửa lỗi nhận diện
F6	Gợi ý nước đi	Gọi engine Pikafish, trả về nước đi tiếp theo (ô gắp, ô đặt, ký hiệu tiếng Việt)
F7	Theo dõi ván	Ghi nhớ diễn biến ván cờ khi robot chơi với người
F8	Giao diện ứng dụng	Tải ảnh lên, hiển thị quân nhận diện, bàn cờ vẽ lại, nước gợi ý, biên bản ván

3.1.2. Yêu cầu phi chức năng

- **Chạy offline** trên robot JetMax: Python 3.6, không Internet, không cài được Ultralytics/PyTorch → chỉ dùng NumPy, OpenCV, TensorRT/ONNX Runtime.
- **Thời gian phản hồi** mỗi lượt vài giây trên Jetson Nano (4 nhân ARM, RAM 4 GB dùng chung).
- **An toàn**: báo "chụp lại" thay vì đi theo một thế cờ sai.
- **Dễ bảo trì**: tách tầng rõ ràng, cấu hình bằng tệp YAML, có bộ kiểm thử tự động và tài liệu API.

3.2. Kiến trúc tổng thể

Hệ thống được thiết kế theo vòng lặp **Nhìn → Hiểu → Quyết định → Hành động** của một robot chơi cờ. Thư viện của đề tài đảm nhiệm ba bước đầu; bước "Hành động" (điều khiển tay máy) do chương trình robot thực hiện dựa trên toạ độ ô mà thư viện trả về.



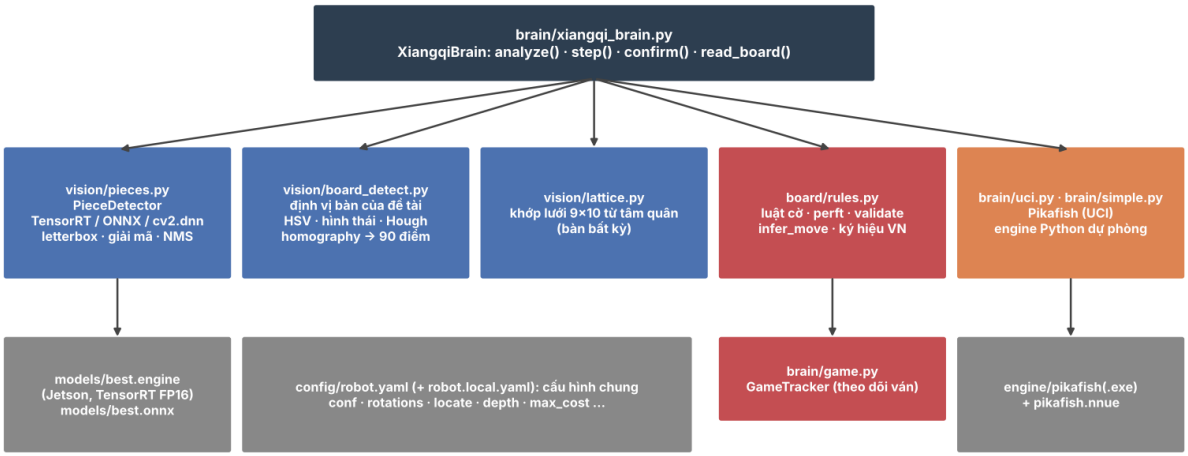
Hình 3. 1. Vòng lặp hoạt động của robot đánh cờ và các giai đoạn thực hiện đề tài

Bảng 3. 2. Bốn tầng xử lý của hệ thống

Tầng	Việc	Mã nguồn	Kỹ thuật chính
1. Nhìn	Ảnh → quân (vị trí + loại)	vision/pieces.py	Tiền xử lý letterbox, YOLO11n, NMS
2. Hiểu	Quân → giao điểm → FEN	vision/board_detect.py, vision/lattice.py	HSV, hình thái học, Hough, homography
3. Kiểm tra	Luật cờ, suy ra nước vừa đi, sửa lỗi	board/rules.py, brain/game.py	Sinh nước hợp lệ, khoảng cách Hamming
4. Quyết định	Tìm nước đi tiếp theo	brain/uci.py	Pikafish (alpha-beta + NNUE)

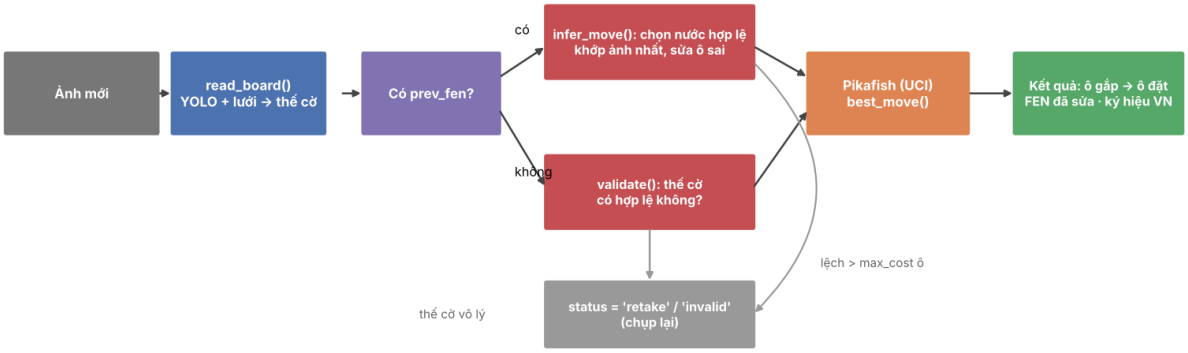
3.3. Thiết kế thư viện

Thư viện được tổ chức thành các gói vision (thị giác), board (luật cờ, hình học bàn cờ), brain (engine, theo dõi ván và lớp giao diện chính XiangqiBrain). Ứng dụng bên ngoài chỉ cần làm việc với một lớp duy nhất.



Hình 3. 2. Sơ đồ các mô-đun của thư viện XiangqiBrain

Luồng xử lý của hàm chính analyze() - *ảnh mới + thế cờ lượt trước → nước đi tiếp theo* - được minh hoạ ở hình dưới. Điểm quan trọng: khi có thế cờ lượt trước, thư viện **không tin tuyệt đối vào ảnh** mà chọn nước hợp lệ khớp ảnh nhất (Chương 6).



Hình 3. 3. Luồng xử lý của hàm XiangqiBrain.analyze()

3.4. Môi trường triển khai

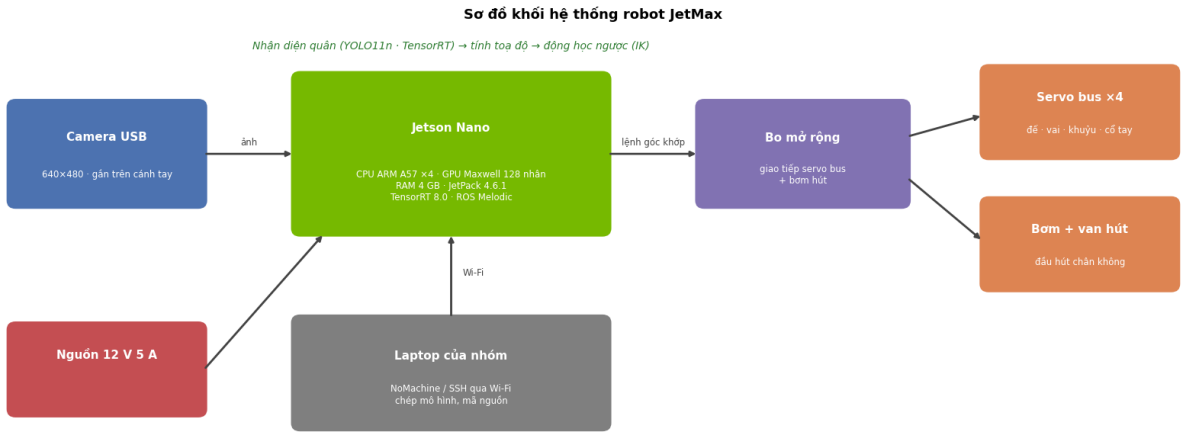
Đề tài sử dụng ba môi trường: **Google Colab** (có GPU, có Internet) để xây dựng dữ liệu, huấn luyện và đóng gói; **robot Hiwonder JetMax** (Jetson Nano) là nền tảng đích; và **laptop Windows** để chạy giao diện demo.



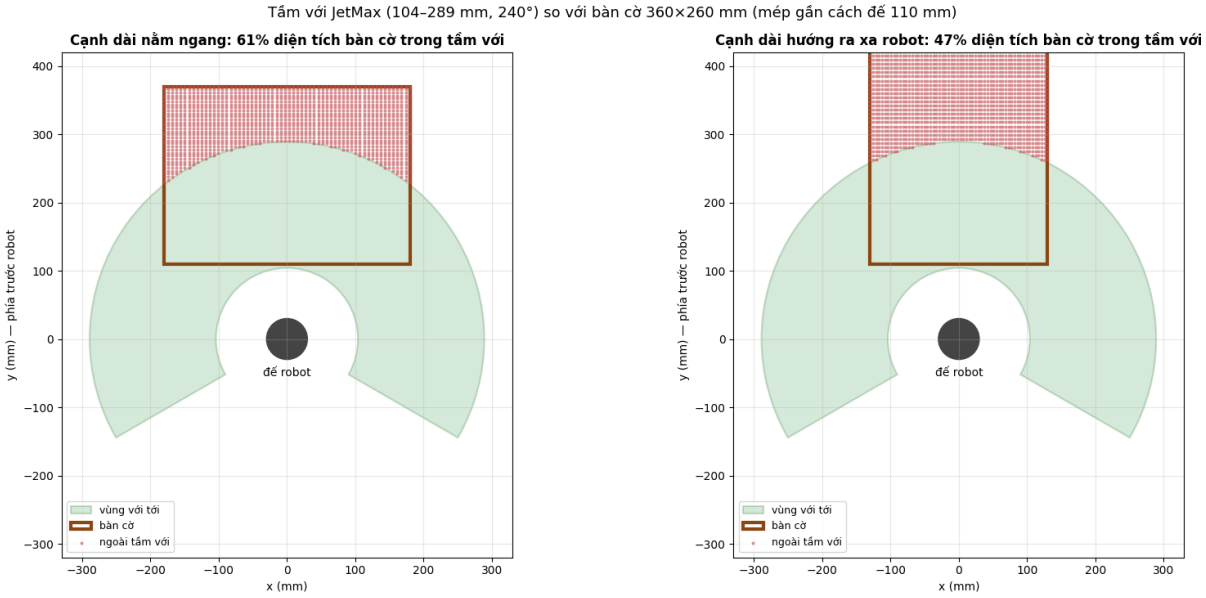
Hình 3. 4. Robot Hiwonder JetMax: tổng thể, cấu tạo và kích thước (nguồn: Hiwonder / OpenELAB)

Bảng 3. 3. Thông số chính của robot JetMax và bộ điều khiển Jetson Nano

Thông số	Giá trị	Ý nghĩa với đề tài
Bậc tự do	4 + 1 (đế, vai, khuỷu + xoay đầu hút)	Đủ để gắp – đặt trên mặt phẳng
Tầm với	104 – 289 mm, góc quét 240°	Nhỏ hơn bàn cờ 360 × 260 mm → giới hạn vùng làm việc
Độ lặp lại	±1 mm	Rất nhỏ so với quân Ø25 mm
Camera	640 × 480, gắn trên cánh tay	Ảnh đầu vào của thư viện
CPU / GPU	ARM Cortex-A57 × 4 / Maxwell 128 nhân	Pikafish chạy CPU, YOLO chạy GPU
RAM	4 GB dùng chung CPU + GPU	Giới hạn bộ nhớ engine và mô hình
Phần mềm	JetPack 4.6.1, CUDA 10.2, TensorRT 8.0.1.6, Python 3.6.9	Không chạy được Ultralytics



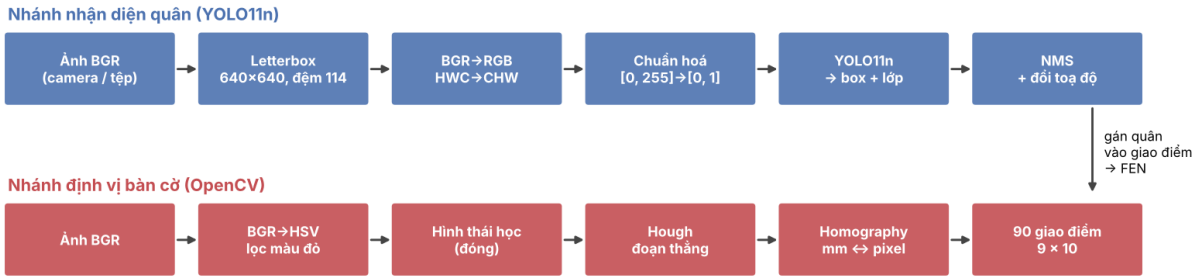
Hình 3. 5. Sơ đồ khối phần cứng hệ thống robot JetMax



Hình 3. 6. Mô phỏng tầm với của JetMax so với bàn cờ 360 × 260 mm (nhìn từ trên)

CHƯƠNG 4. XÂY DỰNG DỮ LIỆU VÀ TIỀN XỬ LÝ ẢNH

Chương này trình bày toàn bộ phần **xử lý ảnh** của đề tài, gồm ba nhóm: (1) xử lý dữ liệu huấn luyện - làm sạch, chuẩn hoá nhãn, khử trùng lặp, chia tập; (2) tiền xử lý ảnh trước khi đưa vào mô hình và tăng cường dữ liệu; (3) chuỗi xử lý ảnh cổ điển để định vị bàn cờ, xác định màu quân và khảo sát các kỹ thuật tăng cường chất lượng ảnh.



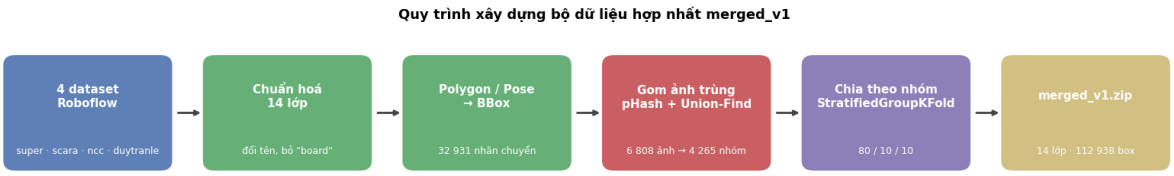
Hình 4. 1. Hai nhánh tiền xử lý ảnh của hệ thống: nhánh nhận diện quân và nhánh định vị bàn cờ

4.1. Nguồn dữ liệu

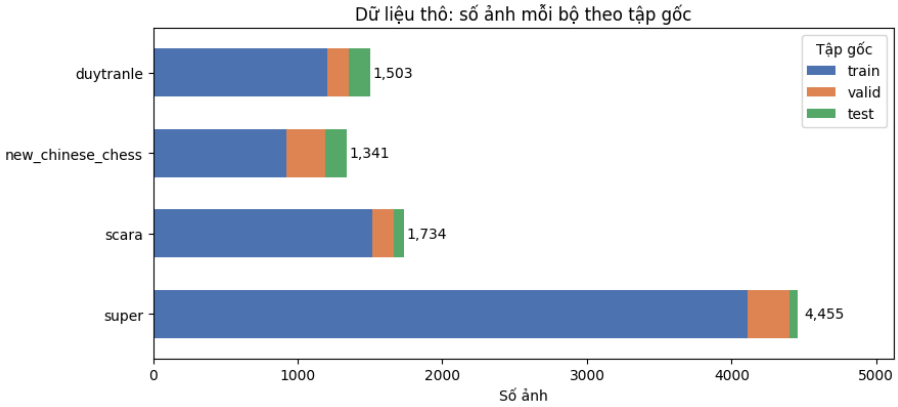
Không có bộ dữ liệu chuẩn đủ lớn cho cờ tướng, và việc tự chụp, gán nhãn hàng chục nghìn quân là không khả thi trong thời gian đề tài. Vì vậy đề tài gộp **4 bộ dữ liệu công khai** trên Roboflow Universe, được chọn theo các tiêu chí: đủ 14 lớp, nhãn chuyển được về hộp bao, tải được theo phiên bản cố định, và **khác nhau** về bộ quân, góc chụp, ánh sáng, nền.

Bảng 4. 1. Bốn bộ dữ liệu nguồn (số ảnh thô theo tập gốc)

Bộ dữ liệu	Đặc điểm	Train	Valid	Test	Tổng
super	Bộ tổng hợp lớn, có lớp "board", một phần nhãn polygon	4 107	291	57	4 455
scara	Ảnh thật của robot SCARA chơi cờ: bàn giữa trận, quân rời	1 518	144	72	1 734
new_chinese_chess	Bộ nhỏ, ít quân mỗi ảnh, nền khác	924	268	149	1 341
duytranle	Nhãn dạng keypoint (pose), nhiều quân trên bàn	1 202	150	151	1 503
Tổng		**7 751**	**853**	**429**	**9 033**



Hình 4. 2. Quy trình xây dựng bộ dữ liệu hợp nhất merged_v1



Hình 4. 3. Số ảnh thô của từng bộ theo tập gốc

4.2. Làm sạch và chuẩn hoá nhãn

Mỗi bộ đặt tên lớp khác nhau (ví dụ Black-Guard, black_advisor, B_Shi…) và dùng các kiểu nhãn khác nhau. Các quy tắc làm sạch:

Bảng 4. 2. Quy tắc làm sạch và chuẩn hoá nhãn

Quy tắc	Ví dụ	Lý do
Viết thường, đổi "-", khoảng trắng → "_"	Black-Guard → black_guard	Thống nhất cách viết
Ánh xạ tên đồng nghĩa về tên chuẩn	guard→advisor, elephant→bishop, chariot→rook, horse→knight, general→king, soldier→pawn	Đưa về một bộ 14 tên
Bỏ dòng nhãn "board", giữ ảnh	2 140 dòng (super), 1 277 dòng (duytranle)	Chỉ nhận diện quân
Bỏ ảnh không còn quân nào	2 137 ảnh (super)	Ảnh rỗng làm lệch dữ liệu
Bỏ cả ảnh có lớp "None"	88 ảnh (new_chinese_chess)	Vật không xác định → nhãn thiếu

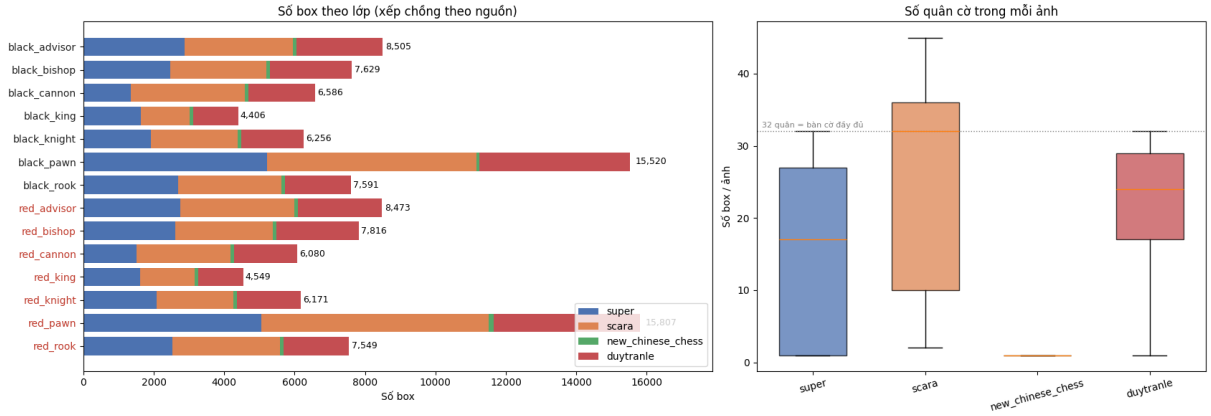
Quy tắc	Ví dụ	Lý do
Polygon / keypoint → hộp bao	1 411 polygon (super), 31 520 pose (duytranle)	Dùng chung định dạng phát hiện
Kẹp hộp vào khung ảnh, bỏ hộp < 0,5%	—	Tránh hộp lỗi làm hỏng hàm mất mát
Gặp tên lớp lạ → dừng và báo lỗi	—	Không để lớp sai lọt qua âm thầm

Với nhãn polygon, hộp bao là hình chữ nhật nhỏ nhất chứa các đỉnh: x_min = min(x_i), x_max = max(x_i) (tương tự với y). Với nhãn pose của bộ duytranle, mỗi dòng có dạng lớp x y w h + keypoint nên chỉ cần lấy 5 giá trị đầu.



Hình 4. 4. Chuyển nhãn polygon (trái) và nhãn pose (phải) về hộp bao

Sau làm sạch còn **6 808 ảnh** với **112 938 quân cờ**. Phân bố lớp không đều: lớp nhiều nhất (red_pawn, 15 807 quân) gấp 3,6 lần lớp ít nhất (black_king, 4 406 quân) – đúng với tỉ lệ quân trong một ván cờ (5 tốt, 1 tướng).



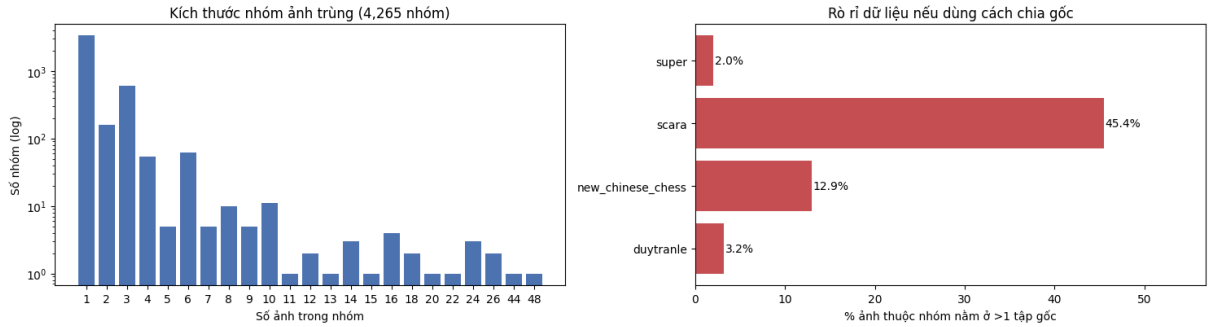
Hình 4. 5. Số quân theo lớp (xếp chồng theo nguồn) và số quân mỗi ảnh của từng bộ

4.3. Phát hiện ảnh trùng bằng băm cảm nhận

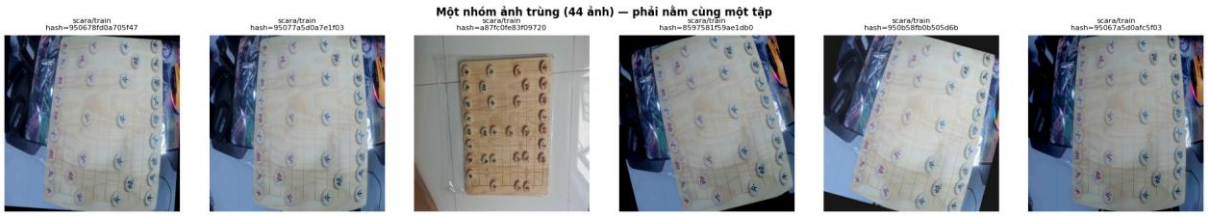
Roboflow thường sinh nhiều phiên bản tăng cường của cùng một ảnh gốc (tên tệp chứa .rf.<mã>), và nhiều ảnh chụp liên tiếp gần như giống hệt nhau. Nếu chia tập ngẫu nhiên, các ảnh gần trùng sẽ rơi vào cả train lẫn test, khiến kết quả kiểm tra **cao giả tạo** (rò rỉ dữ liệu).

Đề tài gom nhóm ảnh trùng bằng hai tiêu chí: (1) cùng tên gốc trước .rf.; (2) **băm cảm nhận pHash** - thu ảnh về 32 × 32 mức xám, biến đổi cosin rời rạc (DCT), lấy 8 × 8 hệ số tần số thấp và so với trung vị để được chuỗi 64 bit; hai ảnh có khoảng cách Hamming giữa hai chuỗi băm nhỏ được xem là gần trùng. Các cặp trùng được hợp nhất thành nhóm bằng cấu trúc **Union-Find**.

Kết quả: 6 808 ảnh thuộc **4 265 nhóm**; nhóm lớn nhất có 48 ảnh (bộ scara). Nếu giữ cách chia gốc của các bộ, **191 nhóm / 1 044 ảnh** bị rò rỉ giữa các tập - riêng bộ scara tới 45,4% số ảnh.



Hình 4. 6. Phân bố kích thước nhóm ảnh trùng và mức rò rỉ nếu dùng cách chia gốc



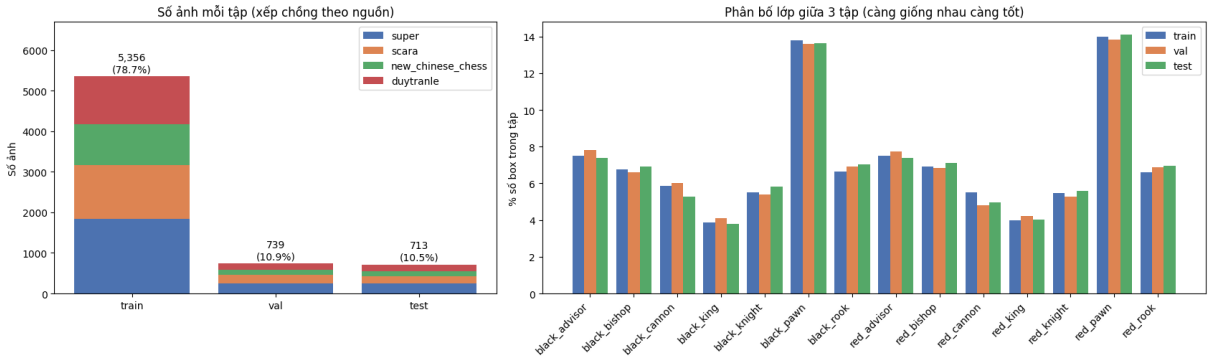
Hình 4. 7. Một nhóm ảnh trùng (44 ảnh) – phải nằm cùng một tập

4.4. Chia tập huấn luyện, kiểm định và kiểm tra

Đề tài dùng **StratifiedGroupKFold** (scikit-learn) để chia đồng thời theo hai điều kiện: mỗi nhóm ảnh trùng chỉ nằm trong **một** tập, và tỉ lệ các lớp giữa các tập gần nhau. Tỉ lệ đạt được ≈ 80/10/10; chênh lệch tỉ lệ lớp lớn nhất giữa train và test chỉ **0,58 điểm %**; kiểm tra sau khi chia xác nhận không nhóm nào bị tách giữa các tập.

Bảng 4. 3. Kết quả chia tập theo nguồn (số ảnh)

Nguồn	Train	Val	Test	Tổng
super	1 837	240	241	2 318
scara	1 335	216	183	1 734
new_chinese_chess	1 000	128	125	1 253
duytranle	1 184	155	164	1 503
Tổng	**5 356 (78,7%)**	**739 (10,9%)**	**713 (10,5%)**	**6 808**



Hình 4. 8. Số ảnh mỗi tập và phân bố lớp giữa ba tập (càng giống nhau càng tốt)



Hình 4. 9. Mẫu của 14 lớp quân cần nhận diện (hàng trên: quân Đen, hàng dưới: quân Đỏ)



Hình 4. 10. Kiểm tra nhãn bằng mắt: ảnh ngẫu nhiên từ các bộ sau khi hợp nhất

4.5. Tiền xử lý ảnh đầu vào mô hình

Mọi ảnh trước khi đưa vào YOLO11n đều đi qua chuỗi tiền xử lý giống hệt nhau khi huấn luyện và khi chạy thật. Trên robot không có Ultralytics nên chuỗi này được **tự cài đặt** trong vision/pieces.py và đã được kiểm chứng cho kết quả trùng với Ultralytics (mục 5.8).

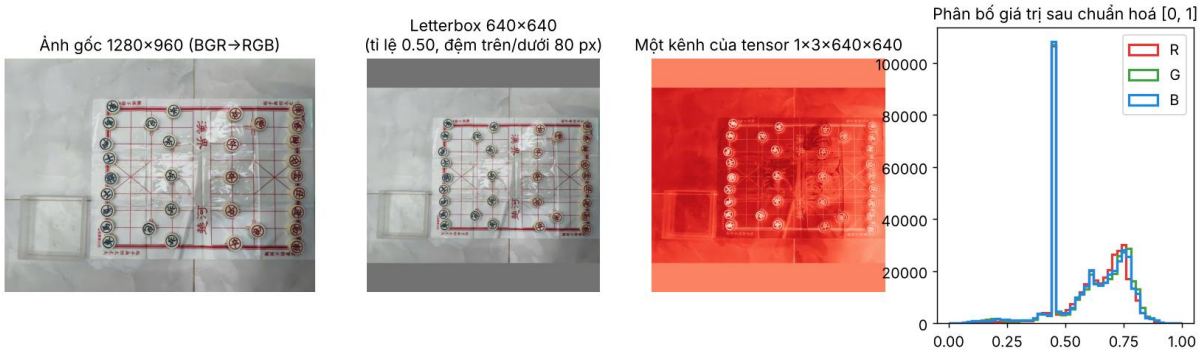
1) **Letterbox:** tính tỉ lệ r = min(640 / H, 640 / W), thu ảnh về (W·r, H·r) bằng nội suy song tuyến tính, đặt vào giữa khung 640 × 640 nền xám 114. Với ảnh 1280 × 960: r = 0,5, ảnh thu về 640 × 480, đệm 80 px trên và dưới. Giữ tỉ lệ giúp quân cờ tròn không bị méo.

2) **Đổi kênh màu BGR → RGB:** OpenCV đọc ảnh theo BGR, mô hình học trên RGB.

3) **Đổi trục HWC → CHW và thêm chiều batch:** tensor có dạng (1, 3, 640, 640).

4) **Chuẩn hoá giá trị:** chia cho 255 để đưa từ [0, 255] về [0, 1], kiểu float32.

5) **Hậu xử lý (sau mô hình):** với mỗi ô trong 8 400 ô lấy lớp có điểm cao nhất, lọc theo ngưỡng tin cậy, NMS loại hộp trùng, rồi đổi toạ độ ngược lại ảnh gốc: x = (x′ − pad_x) / r.

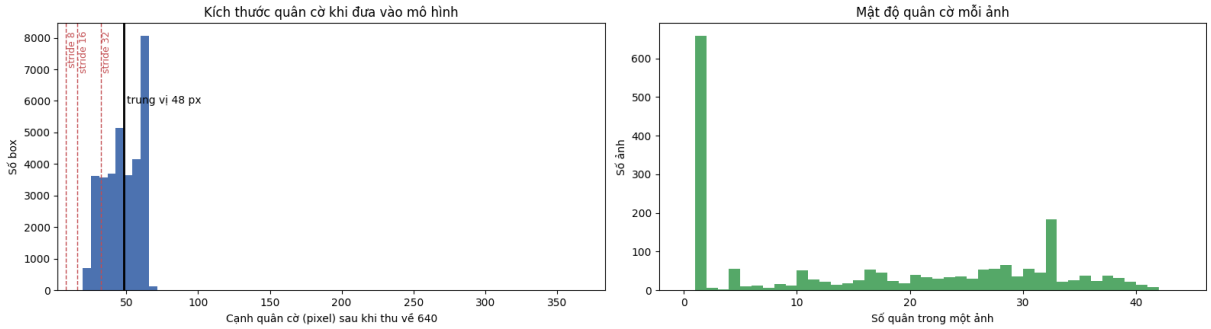


Hình 4. 11. Tiền xử lý ảnh đầu vào: ảnh gốc, letterbox 640 × 640, tensor một kênh và phân bố giá trị sau chuẩn hoá

```
def letterbox(img, size=640):
    h, w = img.shape[:2]
    r = min(size / h, size / w)                    # tỉ lệ thu nhỏ, giữ khung hình
    nw, nh = round(w * r), round(h * r)
    px, py = (size - nw) // 2, (size - nh) // 2       # đệm hai bên
    canvas = np.full((size, size, 3), 114, np.uint8)
    canvas[py:py + nh, px:px + nw] = cv2.resize(img, (nw, nh),
interpolation=cv2.INTER_LINEAR)
    blob = cv2.cvtColor(canvas,
cv2.COLOR_BGR2RGB).astype(np.float32).transpose(2, 0, 1)[None] / 255.0
    return blob, r, px, py
```

4.5.1. Lựa chọn kích thước ảnh đầu vào

Phân tích 33 016 quân từ 2 000 ảnh huấn luyện sau khi thu về 640: cạnh quân trung vị **48,1 px**, chỉ 1,6% quân nhỏ hơn 24 px và không có quân nhỏ hơn 16 px. Ở stride 8 (lưới 80 × 80), một quân 48 px trải trên khoảng 6 ô đặc trưng – đủ chi tiết để đọc nét chữ Hán. Vì vậy chọn imgsz = 640: lớn hơn thì chậm trên Jetson, nhỏ hơn thì mất chi tiết chữ.



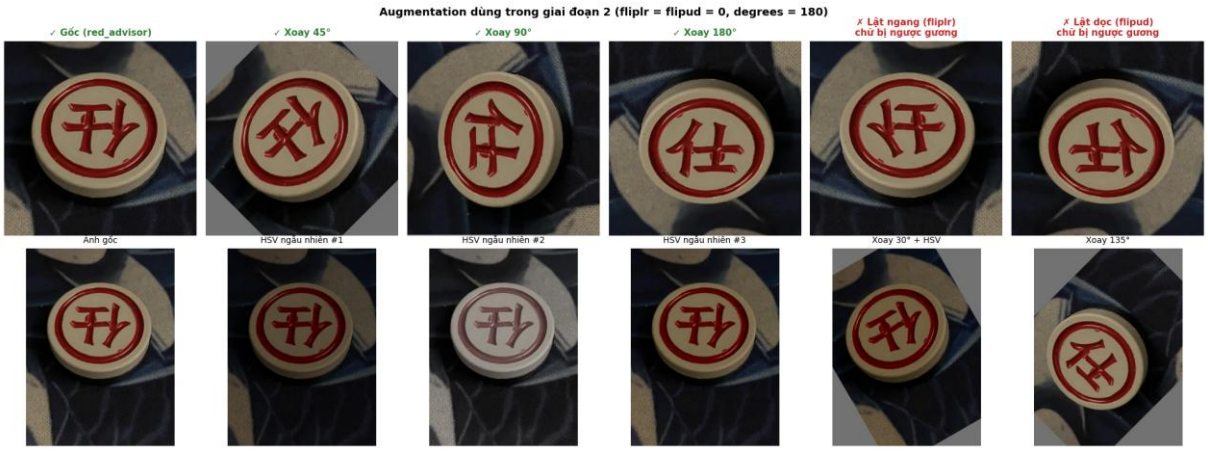
Hình 4. 12. Kích thước quân cờ sau khi đưa về 640 px và mật độ quân trong mỗi ảnh

4.6. Tăng cường dữ liệu

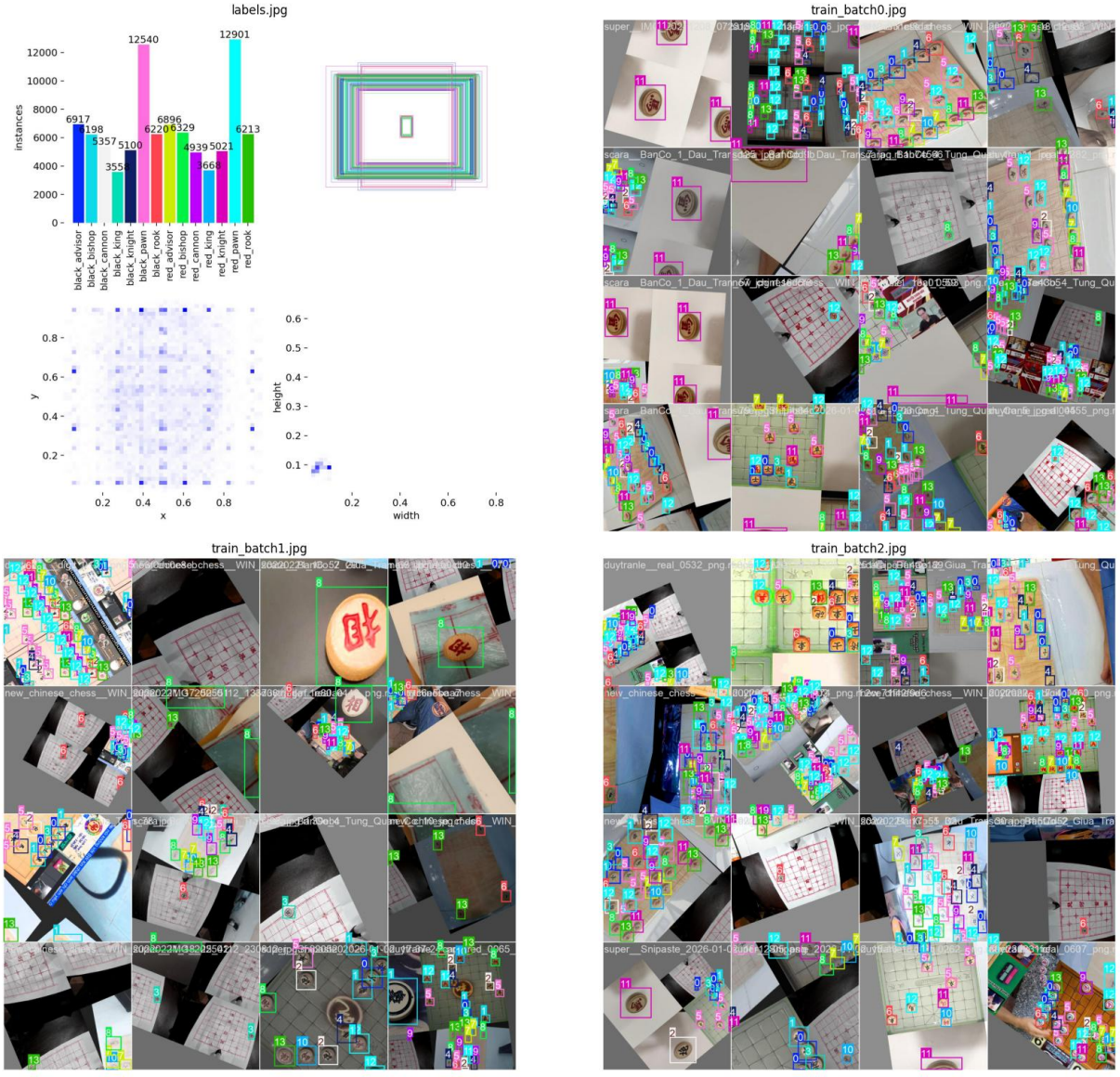
Tăng cường dữ liệu do Ultralytics thực hiện ngẫu nhiên mỗi lần nạp ảnh; đề tài lựa chọn tham số dựa trên đặc thù của quân cờ:

Bảng 4. 4. Các phép tăng cường dữ liệu và lý do lựa chọn

Phép biến đổi	Tham số	Lý do
Xoay ngẫu nhiên	degrees = 180 (giai đoạn 2)	Quân trên bàn / trong khay nằm theo mọi hướng; xoay không làm thay đổi chữ
Lật ngang / lật dọc	fliplr = flipud = 0 (TẮT)	Lật biến chữ Hán thành ảnh gương – chữ không có thật, dạy sai mô hình
Đổi màu HSV	hsv_h 0,015 · hsv_s 0,7 · hsv_v 0,4	Chịu được ánh sáng, màu đèn khác nhau; hsv_h nhỏ để đỏ vẫn là đỏ
Co giãn, dịch	scale 0,5 · translate 0,1	Quân to/nhỏ khác nhau theo khoảng cách camera
Mosaic	mosaic 1,0; tắt ở 10 epoch cuối	Ghép 4 ảnh: nhiều ngữ cảnh, nhiều quân nhỏ; tắt cuối để quen ảnh thật
Xoá ngẫu nhiên	erasing 0,4	Mô phỏng quân bị che một phần



Hình 4. 13. Minh hoạ tăng cường: xoay và đổi màu được dùng; lật ảnh bị loại vì tạo chữ ngược gương



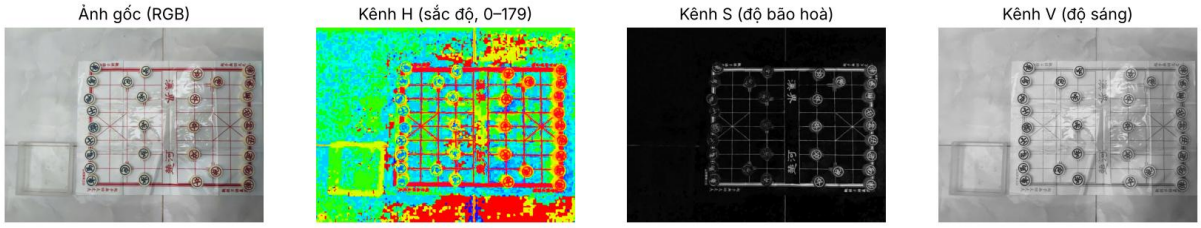
Hình 4. 14. Thống kê nhãn và các lô ảnh huấn luyện sau tăng cường (mosaic, xoay, đổi màu) do Ultralytics sinh ra

4.7. Định vị bàn cờ bằng xử lý ảnh cổ điển

Với bàn cờ của đề tài (tấm nhựa 36 × 26 cm, đường kẻ màu đỏ), mô-đun vision/board_detect.py tìm 90 giao điểm **không cần marker** qua chuỗi xử lý ảnh sau. Các hình trong mục này được tạo từ ảnh chụp thật của bàn cờ (samples/start_position.jpg, 1280 × 960) bằng đúng mã nguồn của đề tài.

4.7.1. Bước 1 - Chuyển sang không gian HSV

Ở kênh H, đường kẻ đỏ và chữ trên quân Đỏ nổi bật thành vùng có sắc độ gần 0, trong khi nền trắng, mặt quân màu kem có độ bão hoà S thấp. Đây là lý do chọn HSV thay vì RGB.

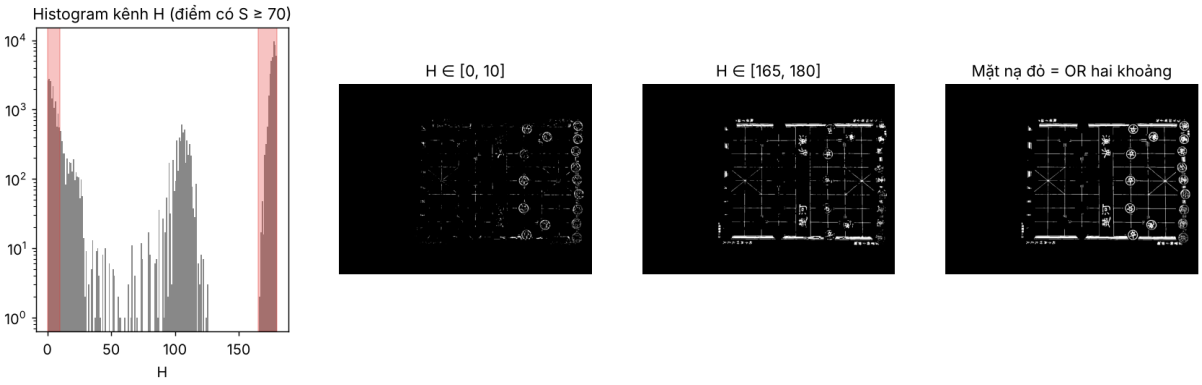


Hình 4. 15. Ảnh bàn cờ thật và ba kênh H, S, V

4.7.2. Bước 2 - Phân ngưỡng màu đỏ

Histogram kênh H (chỉ xét điểm có S ≥ 70) có hai cụm ở hai đầu thang, đúng với tính chất "vòng tròn" của sắc độ. Mặt nạ đỏ là hợp của hai khoảng:

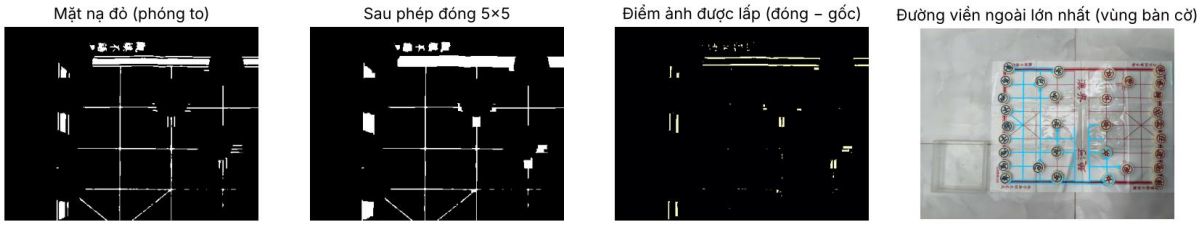
$$mask = inRange(HSV, (0, 70, 60), (10, 255, 255)) \ OR \ inRange(HSV, (165, 70, 60), (180, 255, 255))$$



Hình 4. 16. Histogram kênh H và mặt nạ đỏ tạo từ hai khoảng sắc độ

4.7.3. Bước 3 - Hình thái học và tìm vùng bàn cờ

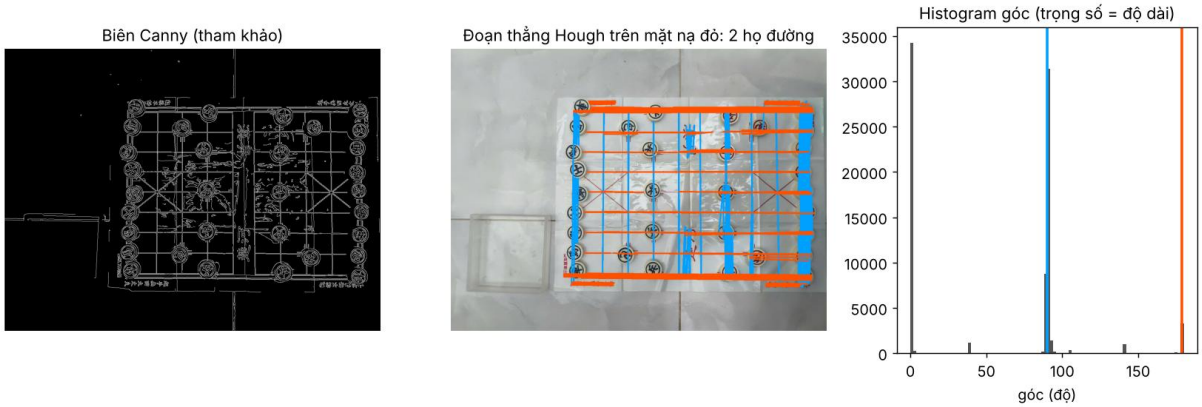
Quân cờ che làm đường kẻ bị đứt đoạn. Phép **đóng** với phần tử cấu trúc hình chữ nhật 5 × 5 nối lại các khe hở nhỏ; sau đó đường viền ngoài lớn nhất cho biết vùng bàn cờ (dùng làm phương án dự phòng khi dò đường thẳng thất bại).



Hình 4. 17. Mặt nạ đỏ trước và sau phép đóng, các điểm ảnh được lấp và đường viền vùng bàn cờ

4.7.4. Bước 4 - Biến đổi Hough tìm hai họ đường lưới

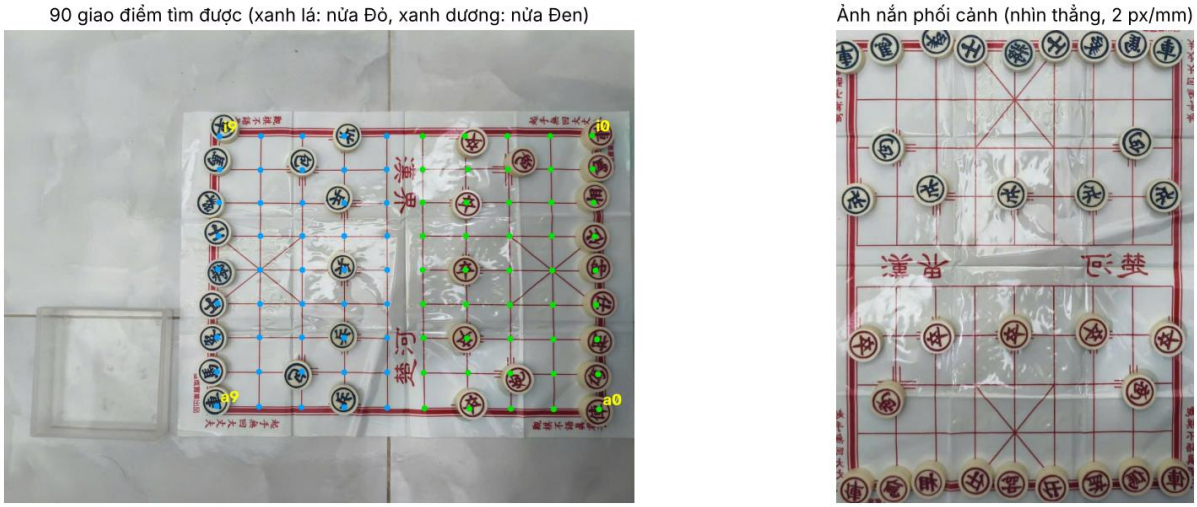
cv2.HoughLinesP được chạy trực tiếp trên mặt nạ đỏ (độ phân giải góc 0,5°, ngưỡng phiếu 4% và độ dài tối thiểu 5% cạnh ảnh). Histogram góc của các đoạn thẳng (trọng số = độ dài) có hai đỉnh cách nhau ≈ 90°: đó là **họ đường dọc** và **họ đường ngang** của lưới. Các đoạn cùng nằm trên một đường được gộp lại, rồi so khớp với mẫu 9 cột × 10 hàng của bàn cờ (khoảng cách thật: cột 27,5 mm, hàng 33,75 mm, sông 30 mm).



Hình 4. 18. Biên Canny (tham khảo), các đoạn thẳng Hough tách thành hai họ đường và histogram góc

4.7.5. Bước 5 - Homography và 90 giao điểm

Giao điểm của các đường đã khớp cho các cặp điểm (mm trên bàn cờ ↔ pixel trên ảnh); homography được ước lượng bằng RANSAC. Sau đó ảnh được nắn về dạng nhìn thẳng, từng đường lưới được dò lại theo từng đoạn ngắn và khớp đường cong bậc 2 (chịu được tấm bàn hơi cong), lặp 2 lần cho hội tụ. Trên ảnh mẫu, sai lệch giữa 90 điểm đo được và mô hình homography chỉ **0,61 mm**.



Hình 4. 19. 90 giao điểm tìm được trên ảnh thật và ảnh bàn cờ sau khi nắn phối cảnh

4.7.6. Bước 6 - Gán quân vào giao điểm

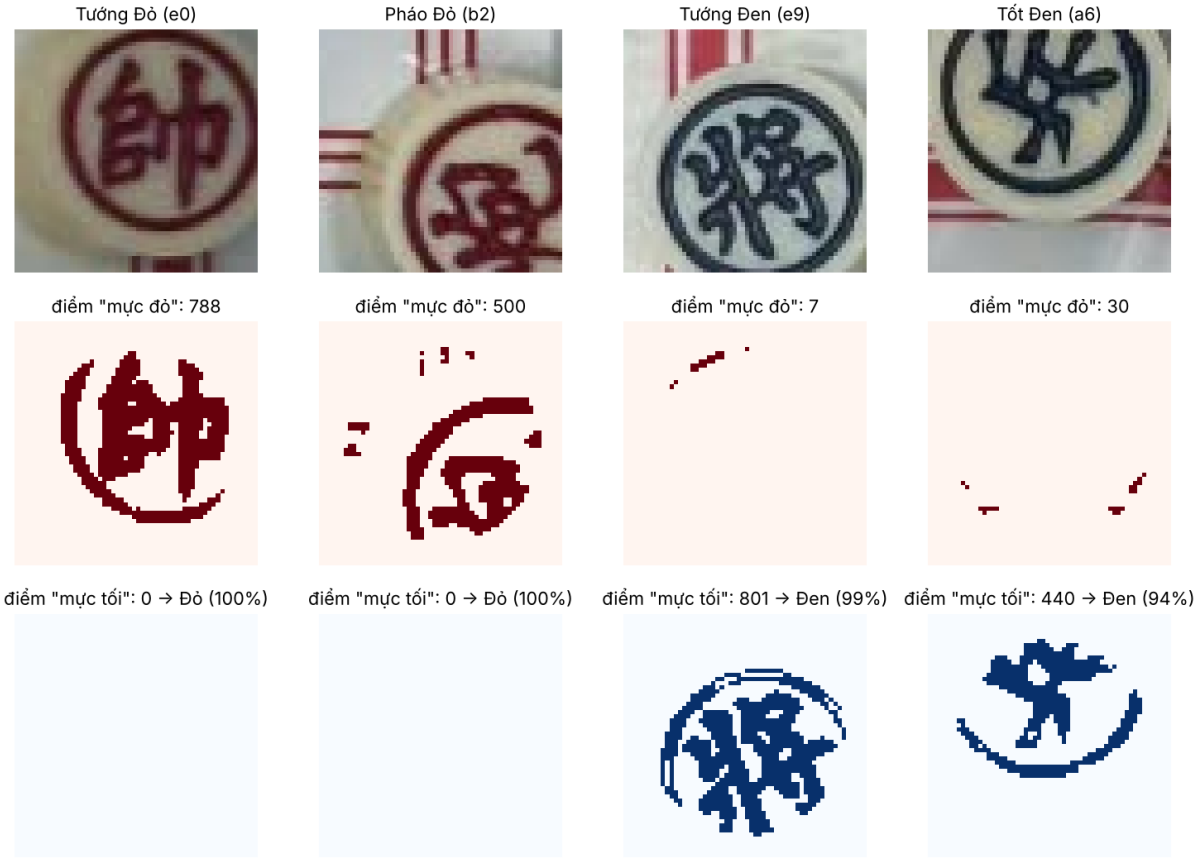
Tâm mỗi quân do YOLO11n trả về được gán vào giao điểm gần nhất nếu khoảng cách nhỏ hơn 0,45 cỡ ô; quân xa hơn được xem là nằm ngoài bàn (vùng chờ, quân đã bị ăn). Nếu hai quân rơi vào cùng một giao điểm, giữ quân có độ tin cậy cao hơn.



Hình 4. 20. Vùng chấp nhận bán kính 0,45 ô quanh mỗi giao điểm

4.8. Xác định màu quân bằng phân ngưỡng HSV

Ngoài mô hình, đề tài dùng thêm một bước xử lý ảnh độc lập để kiểm tra **màu** quân (đỏ / đen): trong hình tròn bán kính 0,8r quanh tâm quân, đếm số điểm "mực đỏ" ($H \leq 10$ hoặc $H \geq 160$, $S \geq 80$, $V \geq 40$) và số điểm "mực tối" (mực xanh đậm $H \in [70, 135]$ hoặc điểm rất tối). Màu có nhiều điểm hơn được chọn. Không dùng tiêu chí "V thấp" đơn thuần vì mặt quân màu kem trong bóng râm cũng tối.



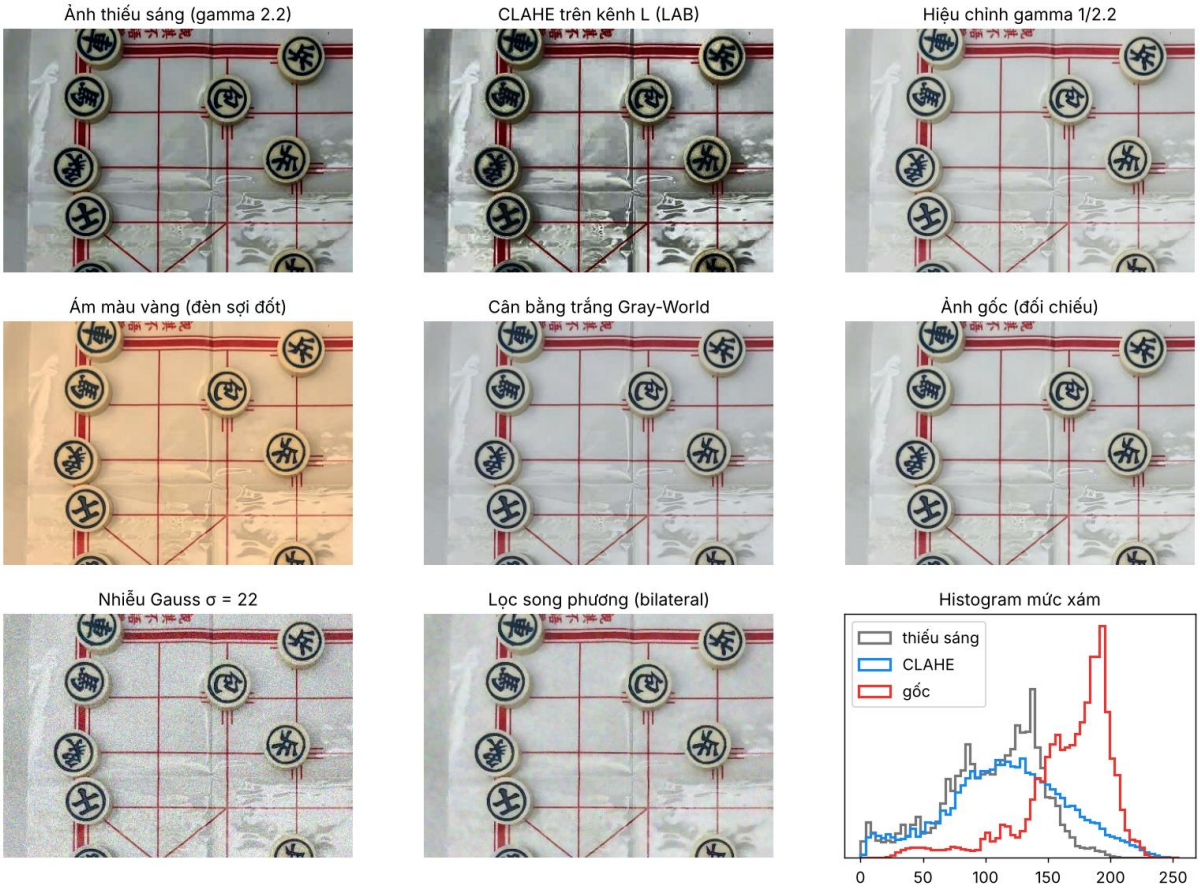
Hình 4. 21. Xác định màu quân: ảnh quân, điểm "mực đỏ" và điểm "mực tối" tìm được bằng phân ngưỡng HSV

4.9. Tăng cường chất lượng ảnh tự động

Ảnh từ camera robot có thể thiếu sáng, ám màu đèn hoặc bị nhiễu. Thư viện có mô-đun **vision/enhance.py** cài đặt bốn kỹ thuật tăng cường chất lượng ảnh dưới đây, chỉ dùng NumPy và OpenCV nên chạy được trên robot. Các thực nghiệm trong mục này nằm ở notebook **training/image_preprocessing.ipynb** (mục 7), trên ảnh suy giảm được mô phỏng từ ảnh chụp thật của đề tài.

Bảng 4. 5. Các kỹ thuật tăng cường chất lượng ảnh trong mô-đun vision/enhance.py

Tình huống suy giảm	Kỹ thuật	Nguyên lý
Thiếu sáng / ngược sáng	CLAHE trên kênh L (LAB), clipLimit 2,0, lưới 8 × 8	Cân bằng histogram cục bộ, giới hạn tương phản để không khuếch đại nhiễu; chỉ tác động độ sáng nên không lệch màu đỏ/đen
Thiếu sáng toàn cục	Hiệu chỉnh gamma (γ = 1/2,2)	Ánh xạ phi tuyến $I_{out} = 255 \cdot (I_{in}/255)^{\gamma}$ bằng bảng tra (LUT)
Ám màu đèn vàng	Cân bằng trắng Gray-World	Nhân mỗi kênh với hệ số để trung bình ba kênh bằng nhau
Nhiễu (ISO cao)	Lọc song phương (bilateral, d = 9)	Trung bình có trọng số theo cả khoảng cách không gian và độ chênh màu → khử nhiễu nhưng giữ biên chữ



Hình 4. 22. Ảnh bị suy giảm (thiếu sáng, ám màu, nhiễu) và kết quả sau các kỹ thuật tăng cường

Nhận xét: CLAHE khôi phục rõ nét chữ và đường kẻ trên ảnh thiếu sáng, histogram mức xám dãn rộng trở lại; Gray-World đưa màu nền về trắng, giúp phân ngưỡng HSV ổn định; lọc song phương giảm nhiễu hạt mà nét chữ vẫn sắc.

Áp dụng trong thư viện – chế độ tự động. Tăng cường mọi ảnh là không nên: ảnh tốt bị biến đổi sẽ khác phân phối dữ liệu huấn luyện và có thể giảm độ chính xác. Vì vậy hàm **enhance()** ở chế độ **auto** (mặc định) đo ảnh trước bằng ba chỉ số, rồi chỉ bật kỹ thuật cần thiết; ảnh tốt đi qua nguyên vẹn nên các kết quả ở Chương 5 và 6 không thay đổi:

– **Độ sáng:** trung bình kênh L của LAB; L < 100 thì bật CLAHE (ảnh tốt của đề tài có L ≈ 173).

– **Độ ám màu:** tỉ số giữa trung bình kênh màu lớn nhất và nhỏ nhất; lớn hơn 1,25 thì bật cân bằng trắng Gray-World (ảnh tốt ≈ 1,02).

– **Mức nhiễu:** độ lệch chuẩn nhiễu ước lượng theo phương pháp Immerkær – tích chập ảnh xám I (kích thước W × H) với mặt nạ Laplace kép M; lớn hơn 4 thì bật lọc

song phương, độ mạnh (sigmaColor = 4σ, giới hạn 40–110) tăng theo mức nhiễu (ảnh tốt ≈ 0,5–0,8):

$$\sigma = \sqrt{(\pi/2)} / (6(W-2)(H-2)) \cdot \Sigma |I * M|, \quad M = [[1, -2, 1], [-2, 4, -2], [1, -2, 1]]$$

Mặc định tăng cường chỉ áp dụng cho **ảnh dùng để định vị bàn cờ** (apply_to: [board]); ảnh đưa vào YOLO11n giữ nguyên, vì mô hình đã được huấn luyện với tăng cường HSV mạnh (hsv_v = 0,4; hsv_s = 0,7) và chưa có ảnh xấu thật từ camera robot để đo. Toàn bộ được cấu hình ở mục **brain.enhance** của config/robot.yaml (off / auto / manual), và kết quả mỗi ảnh (số đo, các bước đã bật) được trả về trong read_board() cũng như hiển thị trên giao diện ứng dụng.

Bảng 4. 6. Chế độ tự động trên ảnh suy giảm mô phỏng: các bước được bật và kết quả định vị bàn cờ trước / sau tăng cường

Ảnh	Độ sáng L	Nhiễu	Ám màu	Bước được bật	Trước tăng cường	Sau tăng cường
3 ảnh mẫu (gốc, bàn trống, quân lộn xộn)	173–175	0,5–0,8	1,01–1,02	không	17 / 19 / 16 đường	giữ nguyên
Tối (gamma 2,2)	112	0,9	1,03	không	15 đường, 0,65 mm	giữ nguyên
Rất tối (gamma 3)	85	0,9	1,04	CLAHE	16 đường, 0,92 mm	14 đường, 0,82 mm
Cực tối (gamma 4)	60	0,8	1,05	CLAHE	12 đường, 0,54 mm	13 đường, 0,76 mm
Ám vàng	170	0,8	1,56	Gray-World	17 đường, 0,56 mm	17 đường, 0,59 mm
Ám vàng mạnh	168	0,8	2,31	Gray-World	17 đường, 0,58 mm	17 đường, 0,57 mm
Nhiễu σ = 8	173	5,4	1,02	Lọc song phương	16 đường, 0,73 mm	17 đường, 0,51 mm
Nhiễu σ = 16	173	10,6	1,02	Lọc song phương	14 đường, 0,84 mm	17 đường, 0,62 mm
Nhiễu σ = 22	174	14,5	1,02	Lọc song phương	MẤT BÀN	16 đường, 0,69 mm
Nhiễu σ = 30	174	19,6	1,02	Lọc song phương	MẤT BÀN	15 đường, 0,79 mm
Nhiễu σ = 40	175	25,6	1,02	Lọc song phương	MẤT BÀN	10 đường, 0,67 mm

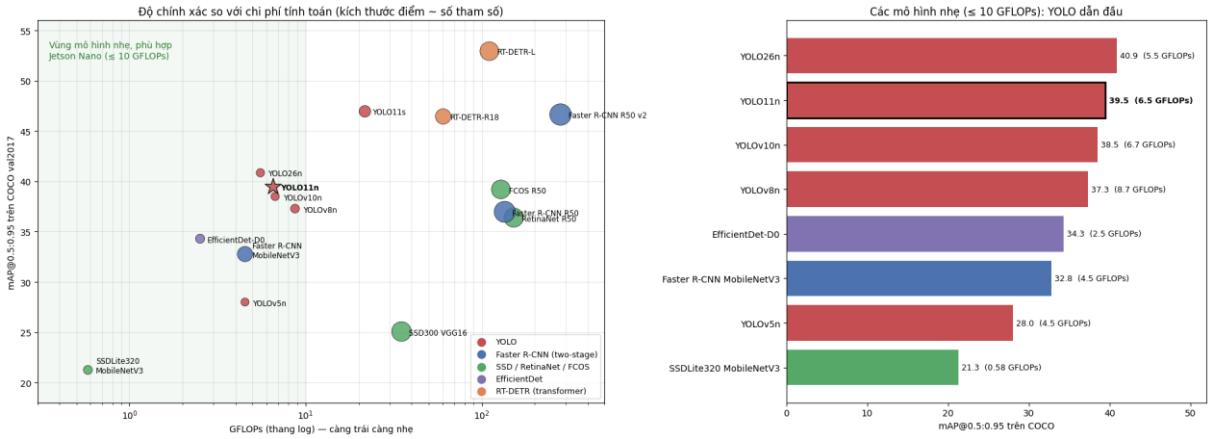
Ảnh	Độ sáng L	Nhiễu	Ám màu	Bước được bật	Trước tăng cường	Sau tăng cường
Rất tối + nhiễu σ = 16	86	10,4	1,04	Lọc + CLAHE	MẤT BÀN	MẤT BÀN

Nhận xét: thiếu sáng và ám màu vốn không làm hỏng việc định vị bàn cờ, vì phân ngưỡng trên kênh H của HSV ít phụ thuộc độ sáng; CLAHE và Gray-World đưa ảnh về gần ảnh tốt và giữ kết quả tương đương. Trường hợp tăng cường **có tác dụng rõ** là nhiễu: từ σ = 22, mặt nạ đỏ lốm đốm làm biến đổi Hough không còn đủ đoạn thẳng và **mất bàn cờ**; sau lọc song phương, bàn cờ **được tìm lại tới σ = 40**. Ảnh vừa rất tối vừa nhiễu vẫn thất bại – khi đó thư viện chuyển sang khớp lưới từ tâm quân hoặc báo chụp lại. Thời gian tăng cường khoảng 30–140 ms cho ảnh 1280 × 960 trên CPU. Bộ kiểm thử tự động (tests/test_brain.py) kiểm tra lại các trường hợp này.

CHƯƠNG 5. HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH NHẬN DẠNG QUÂN CỜ

5.1. Lựa chọn mô hình

Tiêu chí lựa chọn: (1) là mô hình **phát hiện đối tượng** (cần vị trí của tới 32 quân trong một ảnh); (2) đủ nhẹ cho Jetson Nano (≲ 10 GFLOPs); (3) xuất được sang ONNX/TensorRT 8.0; (4) có công cụ huấn luyện hoàn chỉnh. Biểu đồ so sánh (số liệu công bố trên COCO) cho thấy họ YOLO dẫn đầu về độ chính xác trong vùng mô hình nhẹ. YOLO11n đạt 39,5 mAP@0.5:0.95 trên COCO với 6,5 GFLOPs - cao hơn YOLOv8n (37,3) và YOLOv5n (28,0) ở cùng mức chi phí, và được thư viện Ultralytics hỗ trợ đầy đủ.



Hình 5. 1. So sánh độ chính xác – chi phí tính toán của các mô hình phát hiện đối tượng (COCO val2017)

5.2. Phương pháp huấn luyện hai giai đoạn

Đề tài dùng **học chuyển giao (transfer learning)** hai giai đoạn: giai đoạn 1 khởi tạo từ trọng số COCO (yolo11n.pt), huấn luyện 100 epoch với xoay nhẹ ±10° (bài toán ban đầu: quân trên băng chuyền, lệch góc nhỏ); giai đoạn 2 tinh chỉnh (fine-tune) từ kết quả giai đoạn 1 với **xoay ±180°** và learning rate nhỏ để mô hình nhận được quân nằm theo mọi hướng mà không quên đặc trưng đã học.



Hình 5. 2. Quá trình huấn luyện hai giai đoạn và triển khai mô hình

Bảng 5. 1. Tham số huấn luyện của hai giai đoạn (ô in đậm là thay đổi)

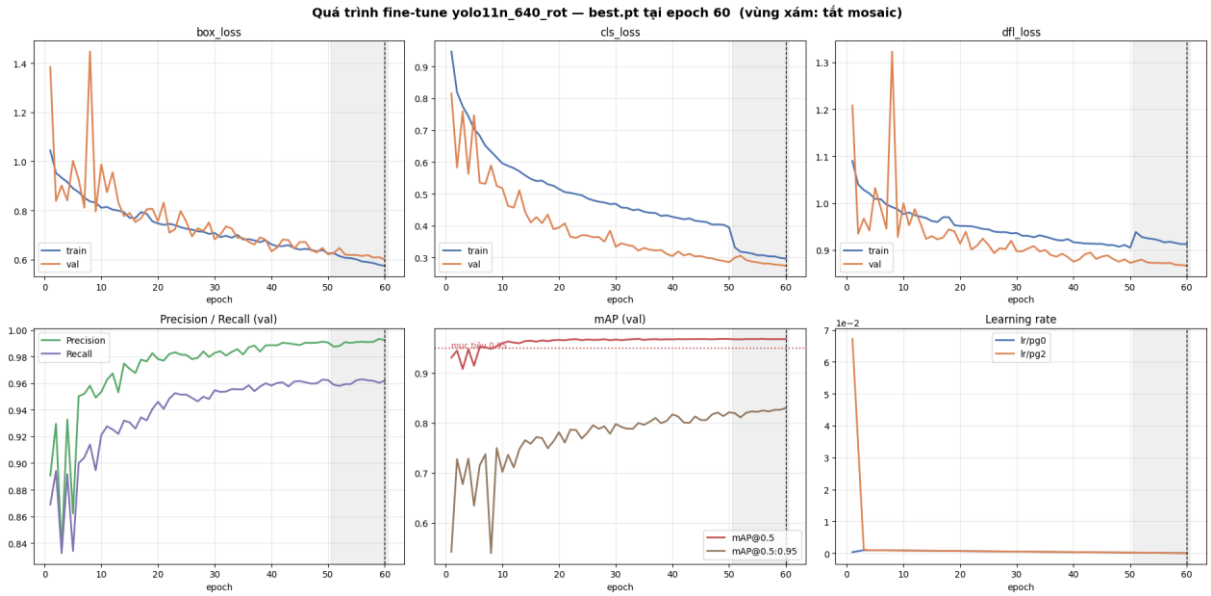
Tham số	Giai đoạn 1	Giai đoạn 2	Lý do thay đổi
Trọng số khởi tạo	yolo11n.pt (COCO)	best.pt giai đoạn 1	Tinh chỉnh từ mô hình đã biết quân cờ
epochs	100	60	Xuất phát từ mô hình tốt → cần ít epoch hơn
optimizer, lr0	auto, 0,01	AdamW, 0,001	LR nhỏ để không phá đặc trưng đã học
degrees	10	180	Quân xoay bất kỳ 0–360°
patience	30	20	Dừng sớm khi đã hội tụ
imgsz, batch	640, 32	640, 32	Vừa bộ nhớ T4 (~6,3 GB); 168 bước/epoch
fliplr, flipud	0, 0	0, 0	Không lật chữ Hán
mosaic, close_mosaic	1,0, 10	1,0, 10	
hsv_h, hsv_s, hsv_v	0,015; 0,7; 0,4	0,015; 0,7; 0,4	
scale, translate	0,5; 0,1	0,5; 0,1	

Mã lệnh huấn luyện giai đoạn 2 với thư viện Ultralytics:

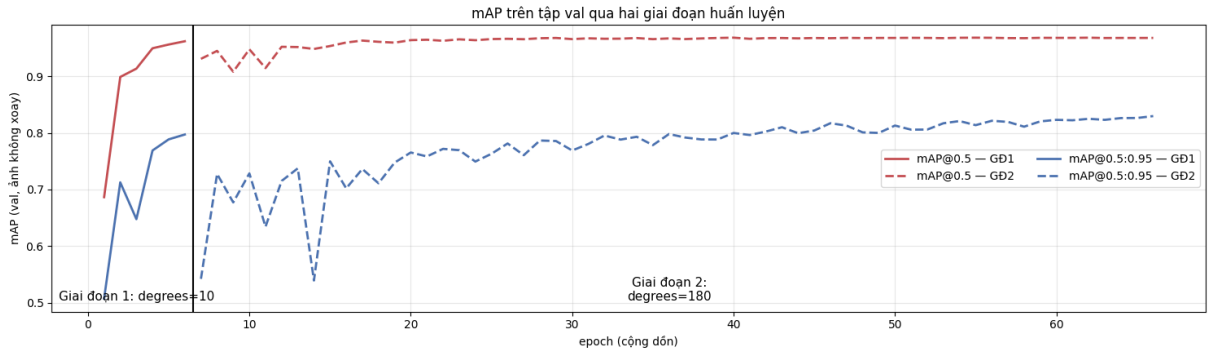
```
from ultralytics import YOLO
model = YOLO('runs/yolo11n_640/weights/best.pt')      # trọng số giai đoạn 1
model.train(data='merged/data.yaml', epochs=60, imgsz=640, batch=32,
            optimizer='AdamW', lr0=0.001, degrees=180, fliplr=0.0, flipud=0.0,
            patience=20, close_mosaic=10, save_period=10, name='yolo11n_640_rot')
```

5.3. Quá trình huấn luyện

Giai đoạn 2 chạy đủ 60 epoch trong **1,69 giờ** trên GPU Tesla T4. Hàm mất mát trên tập train và val đều giảm đều, không có dấu hiệu quá khớp; ở 10 epoch cuối (vùng xám - tắt mosaic) loss giảm thêm một bậc. mAP@0.5 trên tập val ổn định quanh 0,97, mAP@0.5:0.95 tiếp tục tăng tới cuối.



Hình 5. 3. Theo dõi quá trình tinh chỉnh: box/cls/dfl loss, Precision – Recall, mAP và learning rate trên tập val



Hình 5. 4. mAP trên tập val qua hai giai đoạn huấn luyện

Bảng 5. 2. Kết quả trên tập val tại epoch tốt nhất (epoch 60)

Precision	Recall	mAP@0.5	mAP@0.5:0.95
0,992	0,962	0,968	0,830

5.4. Kết quả trên tập kiểm tra

Mô hình được đánh giá trên **713 ảnh kiểm tra (11 950 quân)** chưa từng dùng khi huấn luyện, bằng chế độ val của Ultralytics:

Bảng 5. 3. Kết quả tổng hợp trên tập kiểm tra

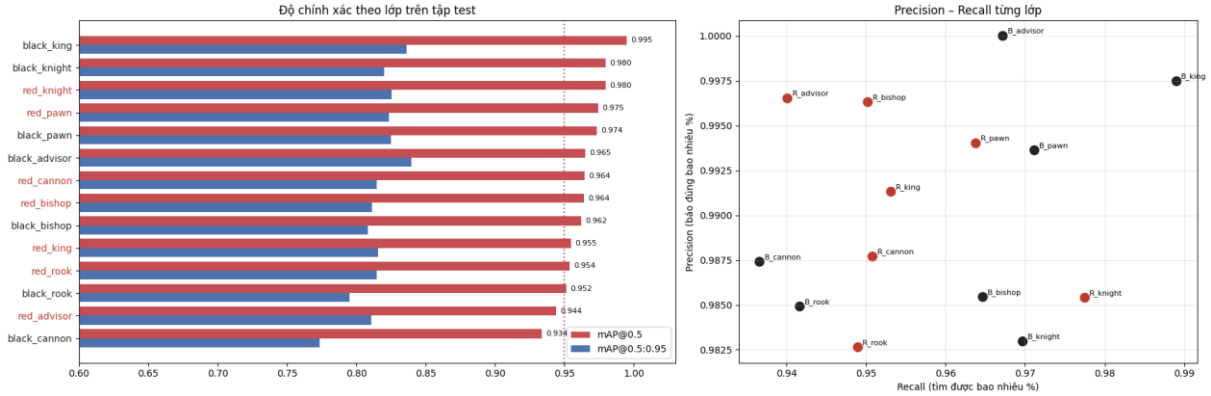
Chỉ số	Giá trị	Mục tiêu
mAP@0.5	0,964	≥ 0,95 ✓
mAP@0.5:0.95	0,815	
Precision	0,990	
Recall	0,959	
Thời gian / ảnh (T4)	1,4 ms tiền xử lý + 4,6 ms suy luận + 2,0 ms hậu xử lý	
Kích thước mô hình	2 600 210 tham số · 6,5 GFLOPs · 5,5 MB	Chạy được trên Jetson Nano ✓

Bảng 5. 4. Kết quả theo từng lớp trên tập kiểm tra

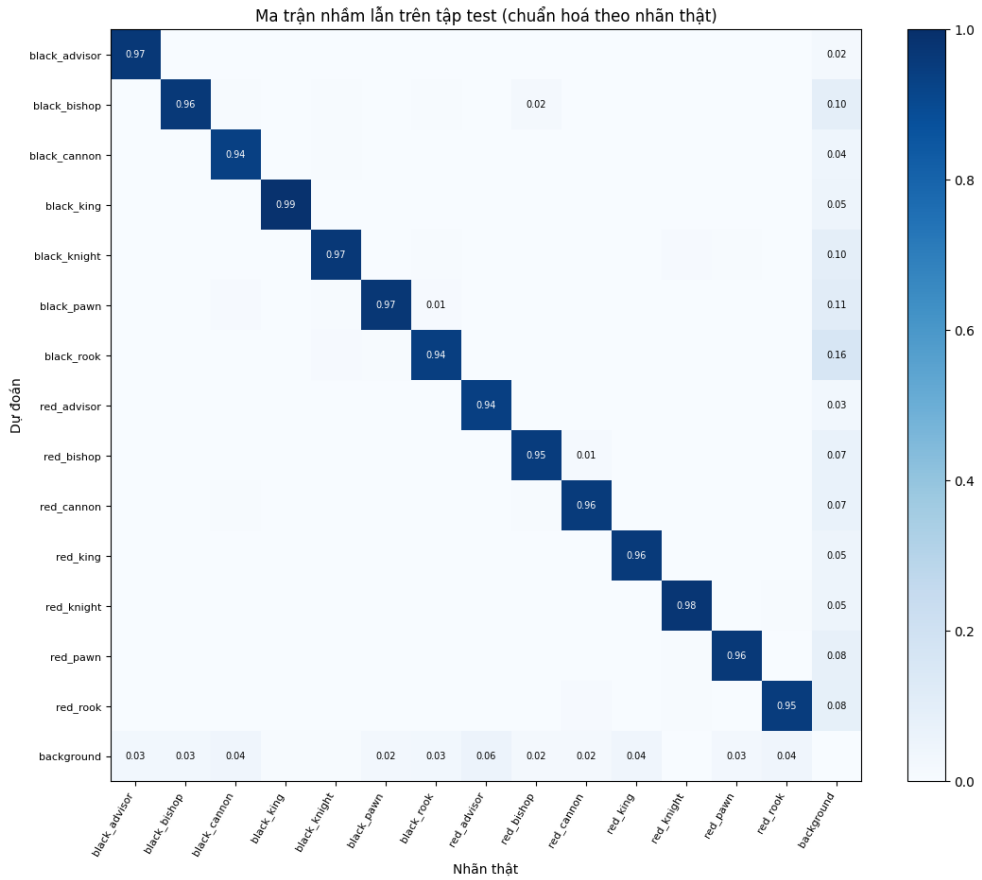
Lớp	Số ảnh	Số quân	P	R	mAP@0.5	mAP@0.5:0.95
black_advisor	449	885	1,000	0,967	0,965	0,840
black_bishop	426	821	0,985	0,965	0,962	0,808
black_cannon	350	631	0,987	0,937	0,934	0,773
black_king	453	456	0,997	0,989	0,995	0,836
black_knight	423	693	0,983	0,970	0,980	0,820
black_pawn	443	1 630	0,994	0,971	0,974	0,825
black_rook	419	840	0,985	0,942	0,952	0,795
red_advisor	435	885	0,997	0,940	0,944	0,811
red_bishop	446	852	0,996	0,950	0,964	0,811
red_cannon	337	591	0,988	0,951	0,964	0,815
red_king	460	479	0,991	0,953	0,955	0,816
red_knight	397	667	0,985	0,978	0,980	0,826

Lớp	Số ảnh	Số quân	P	R	mAP@0.5	mAP@0.5:0.95
red_pawn	463	1 685	0,994	0,964	0,975	0,824
red_rook	410	835	0,983	0,949	0,954	0,814
Tất cả	**713**	**11 950**	**0,990**	**0,959**	**0,964**	**0,815**

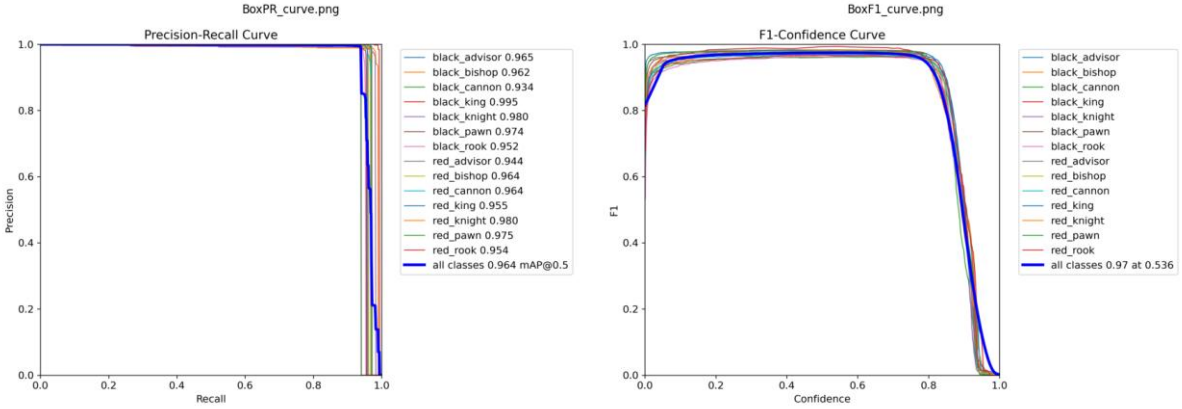
Nhận xét: Precision (0,990) cao hơn Recall (0,959) – mô hình **ít báo nhầm**, lỗi chủ yếu là **bỏ sót** quân. Với robot, bỏ sót an toàn hơn báo nhầm: quân bị sót có thể được nhận ở lần chụp sau, và luật cờ (Chương 6) bù lại được. Lớp yếu nhất là black_cannon (mAP@0.5 = 0,934). Ma trận nhầm lẫn cho thấy các lớp hầu như không bị nhầm lẫn với nhau (đường chéo ≥ 0,94); phần sai lệch nằm ở cột "background" – tức bỏ sót: red_advisor 5,6%, black_cannon 4,3%, red_king 4,0%, red_rook 4,0%.



Hình 5. 5. mAP theo từng lớp và quan hệ Precision – Recall của từng lớp trên tập kiểm tra



Hình 5. 6. Ma trận nhầm lẫn trên tập kiểm tra (chuẩn hoá theo nhãn thật)



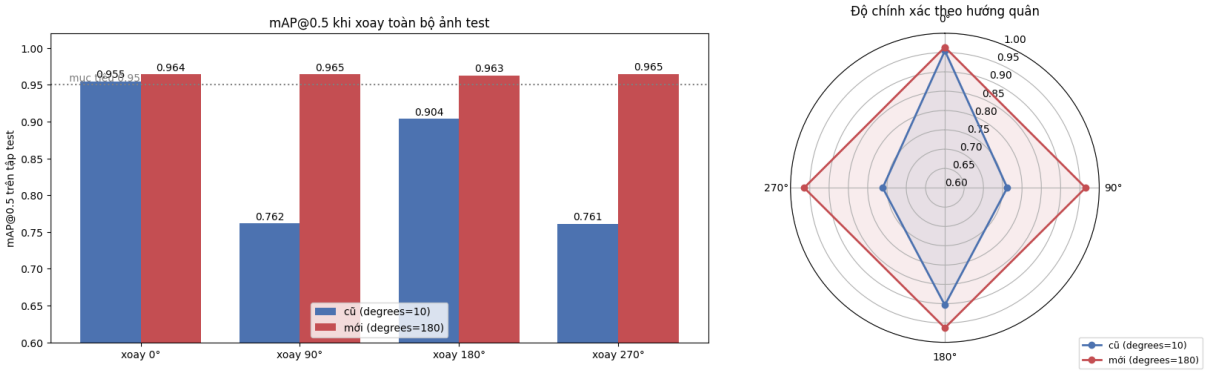
Hình 5. 7. Đường cong Precision – Recall và F1 – Confidence của 14 lớp

5.5. Độ bền khi quân cờ bị xoay

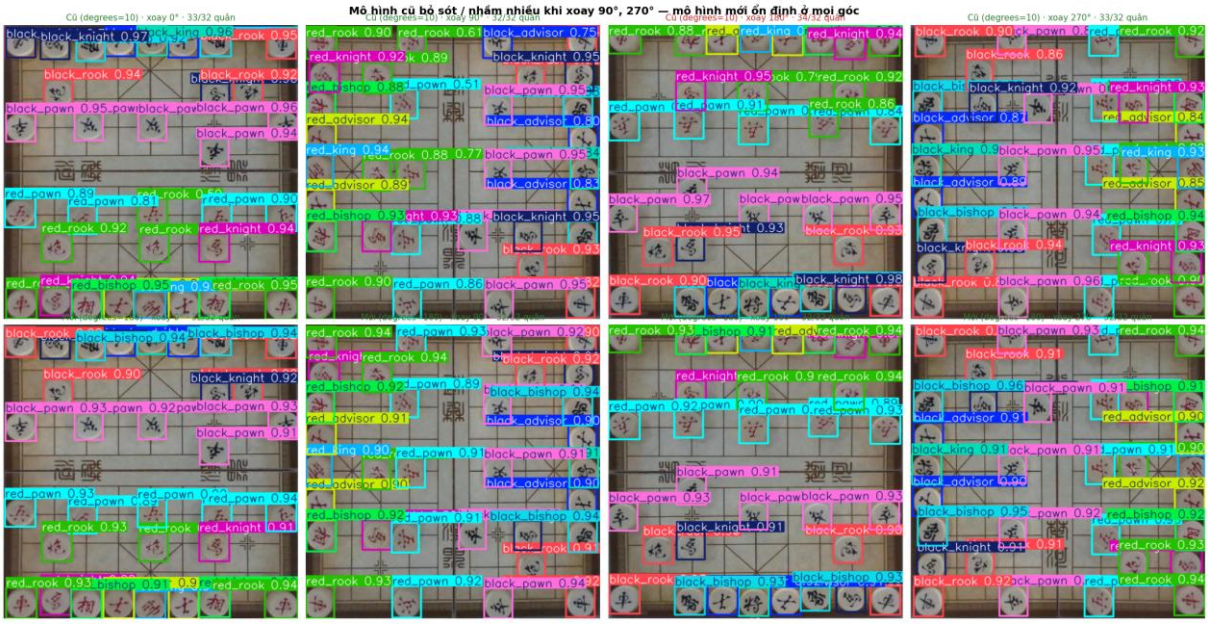
Để kiểm tra khả năng nhận quân nằm theo mọi hướng, toàn bộ 713 ảnh kiểm tra được xoay 90°, 180°, 270° (kèm biến đổi toạ độ nhãn tương ứng) rồi đánh giá lại cả mô hình giai đoạn 1 (degrees = 10) và mô hình cuối (degrees = 180):

Bảng 5. 5. mAP@0.5 trên tập kiểm tra khi xoay toàn bộ ảnh

Mô hình	0°	90°	180°	270°
Giai đoạn 1 (degrees = 10)	0,955	0,762	0,904	0,761
Giai đoạn 2 (degrees = 180)	0,964	0,965	0,963	0,965



Hình 5. 8. Độ chính xác theo góc xoay của hai mô hình



Hình 5. 9. Cùng một ảnh kiểm tra xoay 4 góc: mô hình cũ bỏ sót / nhầm nhiều ở 90° và 270°, mô hình mới ổn định

5.6. Ví dụ dự đoán



Hình 5. 10. So sánh nhãn thật (trái) và dự đoán của mô hình (phải) trên một ảnh: 16/16 quân đúng (TP = 16, FP = 0, FN = 0)



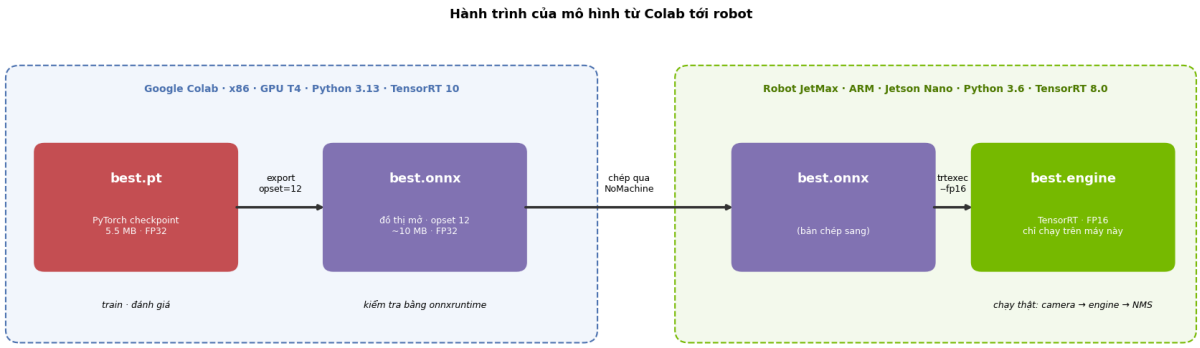
Hình 5. 11. Dự đoán trên ảnh bàn cờ chụp nghiêng, có lóa sáng



Hình 5. 12. Dự đoán trên ảnh bàn cờ thật nằm nghiêng (bộ scara)

5.7. Xuất mô hình sang ONNX và kiểm tra

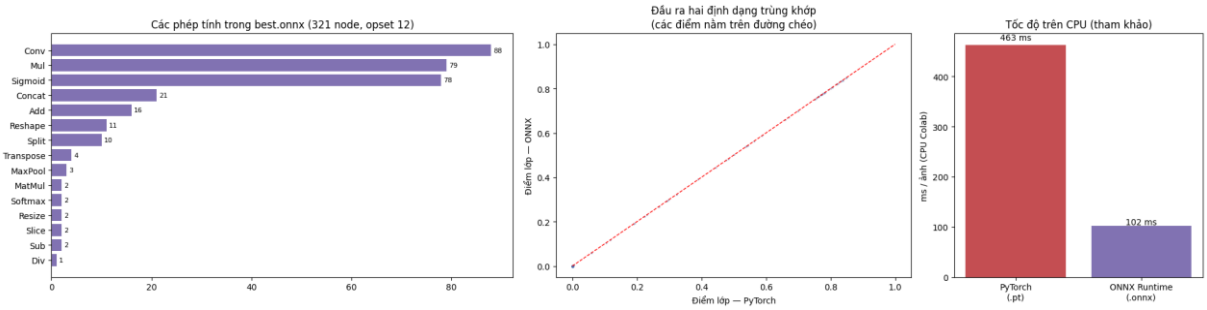
Trên robot không chạy được PyTorch/Ultralytics nên mô hình phải đi qua ba dạng: best.pt (PyTorch, huấn luyện) → best.onnx (định dạng mở, opset 12, xuất trên Colab) → best.engine (TensorRT FP16, build trên chính Jetson Nano bằng trtexec).



Hình 5. 13. Hành trình của mô hình từ Colab tới robot

Kiểm tra trước khi mang sang robot: so sánh trực tiếp tensor đầu ra (1, 18, 8400) của PyTorch và ONNX trên cùng một ảnh - sai lệch tuyệt đối lớn nhất $\mathbf{1{,}07 \cdot 10^{-3}}$, trung bình $\mathbf{5{,}63 \cdot 10^{-6}}$, lớp dự đoán trùng nhau **100%**. Đồ thị ONNX gồm 321 nút, chủ yếu Conv, Mul, Sigmoid (SiLU), Concat - đều được TensorRT 8.0 hỗ trợ. Hậu xử lý tự viết

(letterbox → giải mã → NMS) cho đúng 32/32 quân như Ultralytics; trên CPU Colab, ONNX Runtime mất khoảng 94 ms/ảnh.



Hình 5. 14. Các phép tính trong best.onnx, so sánh đầu ra PyTorch – ONNX và tốc độ trên CPU



Hình 5. 15. Kết quả PyTorch (trái) và ONNX + NMS tự viết (phải) trên cùng một ảnh: đều 32 quân

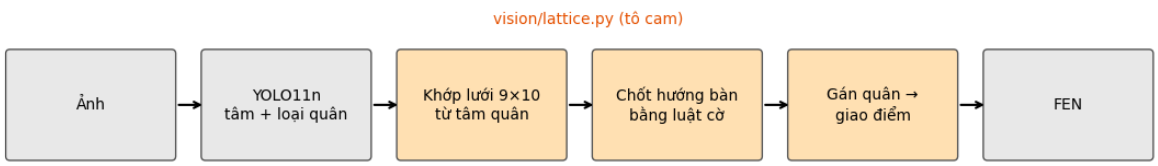
CHƯƠNG 6. TỪ ẢNH ĐẾN THẾ CỜ VÀ NƯỚC ĐI

mAP đo độ chính xác trên **từng quân**, nhưng engine cần **cả thế cờ** đúng: chỉ sai 1 trong 32 quân là FEN đã sai. Chương này trình bày cách chuyển ảnh thành thế cờ, cách đánh giá trọn chuỗi ảnh → FEN, và cách dùng luật cờ để phát hiện, sửa lỗi nhận dạng.

6.1. Khớp lưới 9 × 10 từ tâm quân

Tập kiểm tra gồm ảnh từ 4 bộ dữ liệu, mỗi bộ một kiểu bàn cờ, màu đường kẻ, góc chụp - nên không thể dùng board_detect.py (chỉ dành cho bàn kẻ đỏ của đề tài). Đề tài xây dựng thuật toán vision/lattice.py dựa trên nhận xét: quân cờ luôn đứng trên giao điểm, nên **tâm các quân nằm trên một lưới đều đã bị biến đổi phối cảnh**. Thuật toán:

1) **Ước lượng hướng trục và cỡ ô:** nối mỗi quân với 8 quân gần nhất; histogram góc của các vectơ cho các hướng trục ứng viên. Độ dài vectơ dọc một trục là bội số nguyên của cỡ ô d, nên chọn d lớn nhất sao cho mọi độ dài ℓ đều gần k·d (k nguyên).
2) **Khớp homography lưới → ảnh và mở rộng dần:** gán toạ độ nguyên cho các quân gần quân "hạt giống", khớp homography, rồi mở rộng bán kính 2,5 → 14 ô và khớp lại – chịu được phối cảnh (ô xa camera nhỏ hơn).
3) **Chọn lưới tốt nhất:** lưới giải thích được nhiều giao điểm có quân nhất; hoà thì chọn ô lớn nhất (lưới nửa ô cũng "khớp" mọi điểm).
4) **Chốt hướng và vị trí bàn bằng luật cờ:** thử 8 phép đối xứng × các vị trí dịch, chỉ giữ cách đặt cùng chiều với ảnh thật (ảnh chụp không bị lật gương), chọn cách cho thế cờ ít vi phạm luật nhất (tướng/sĩ trong cung, tượng không qua sông…) và quân Đỏ ở hàng 0–4.
5) **Quân ngoài khung 9 × 10** hoặc lệch giao điểm > 0,35 ô được xem là để ngoài bàn.



Hình 6. 1. Luồng xử lý ảnh → FEN bằng khớp lưới từ tâm quân

Kiểm chứng trên dữ liệu tổng hợp (biết trước đáp án): sinh thế cờ từ ván ngẫu nhiên, chiếu các giao điểm có quân lên ảnh với góc xoay bất kỳ, phối cảnh ngẫu nhiên và nhiễu vị trí 5% cỡ ô.

Bảng 6. 1. Tỉ lệ khôi phục đúng toàn bộ thế cờ trên dữ liệu tổng hợp

Số quân trên bàn	Gần nhìn thẳng (p = 0,1)	Chụp xiên (p = 0,3)	Số thế cờ
10 – 19 quân	93,8%	89,1%	113 / 110
≥ 20 quân	96,0%	98,4%	125 / 128

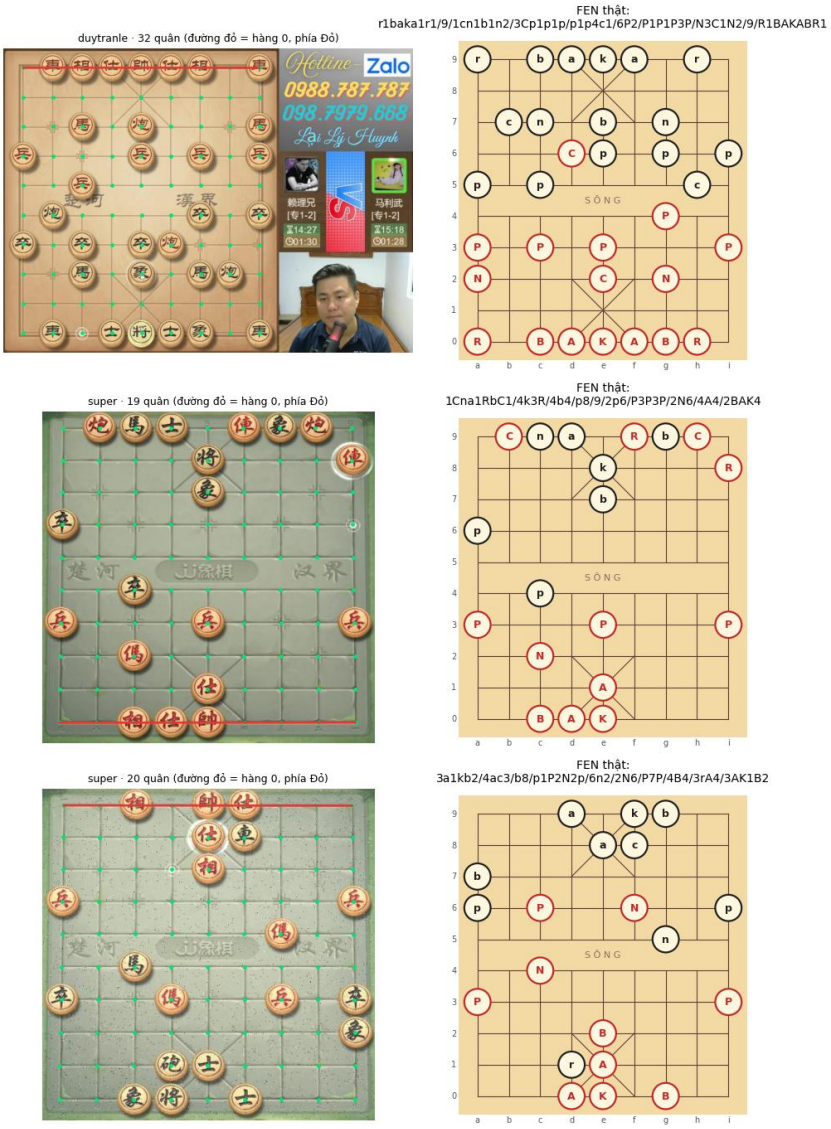
6.2. Đánh giá chuỗi ảnh → FEN trên tập kiểm tra

6.2.1. Tạo FEN thật cho ảnh kiểm tra

Tập dữ liệu chỉ có nhãn hộp bao, không có FEN. FEN thật được tạo bằng cách chạy thuật toán khớp lưới lên **tâm các hộp nhãn** (vị trí và loại quân do người gán nhãn), rồi chỉ giữ ảnh có kết quả đáng tin. Do đó tập đánh giá thiên về ảnh "dễ khớp lưới" hơn tập kiểm tra gốc.

Bảng 6. 2. Các bước lọc tạo tập đánh giá ảnh → FEN

Bước lọc	Số ảnh	Tỉ lệ
Ảnh trong tập kiểm tra	713	100,0%
Có ≥ 10 quân (ảnh có bàn cờ)	451	63,3%
Khớp được lưới	451	63,3%
≥ 90% quân nằm trên giao điểm	342	48,0%
Thế cờ hợp lệ, cách đặt không mơ hồ → tập đánh giá	229	32,1%



Hình 6. 2. Lưới khớp được (chấm xanh) trên ảnh kiểm tra và FEN thật tương ứng

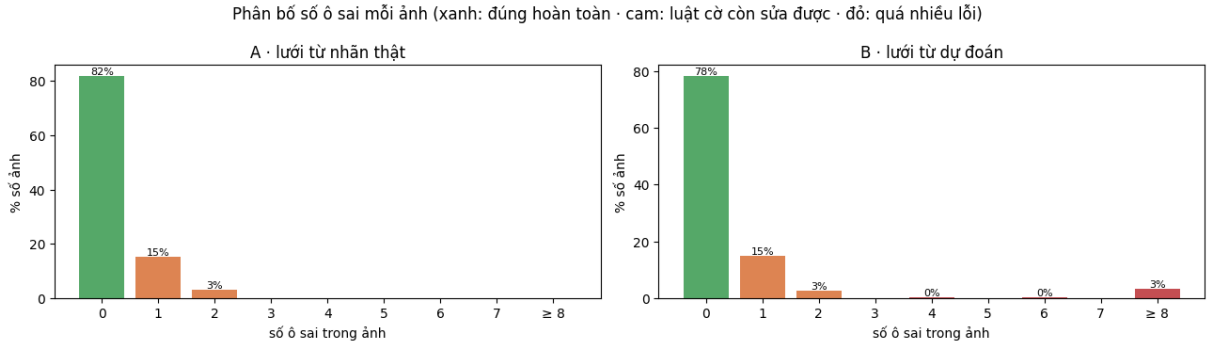
6.2.2. Kết quả

Hai thiết lập được so sánh: **A** - lưới lấy từ nhãn thật (chỉ đo lỗi của mô hình nhận diện quân, tương ứng robot đã định vị chính xác bàn cờ); **B** - lưới khớp từ chính các quân mô hình dự đoán (đo toàn bộ chuỗi ảnh → FEN, không dùng nhãn).

Bảng 6. 3. Kết quả ảnh → FEN trên 229 ảnh đánh giá (ngưỡng tin cậy 0,5)

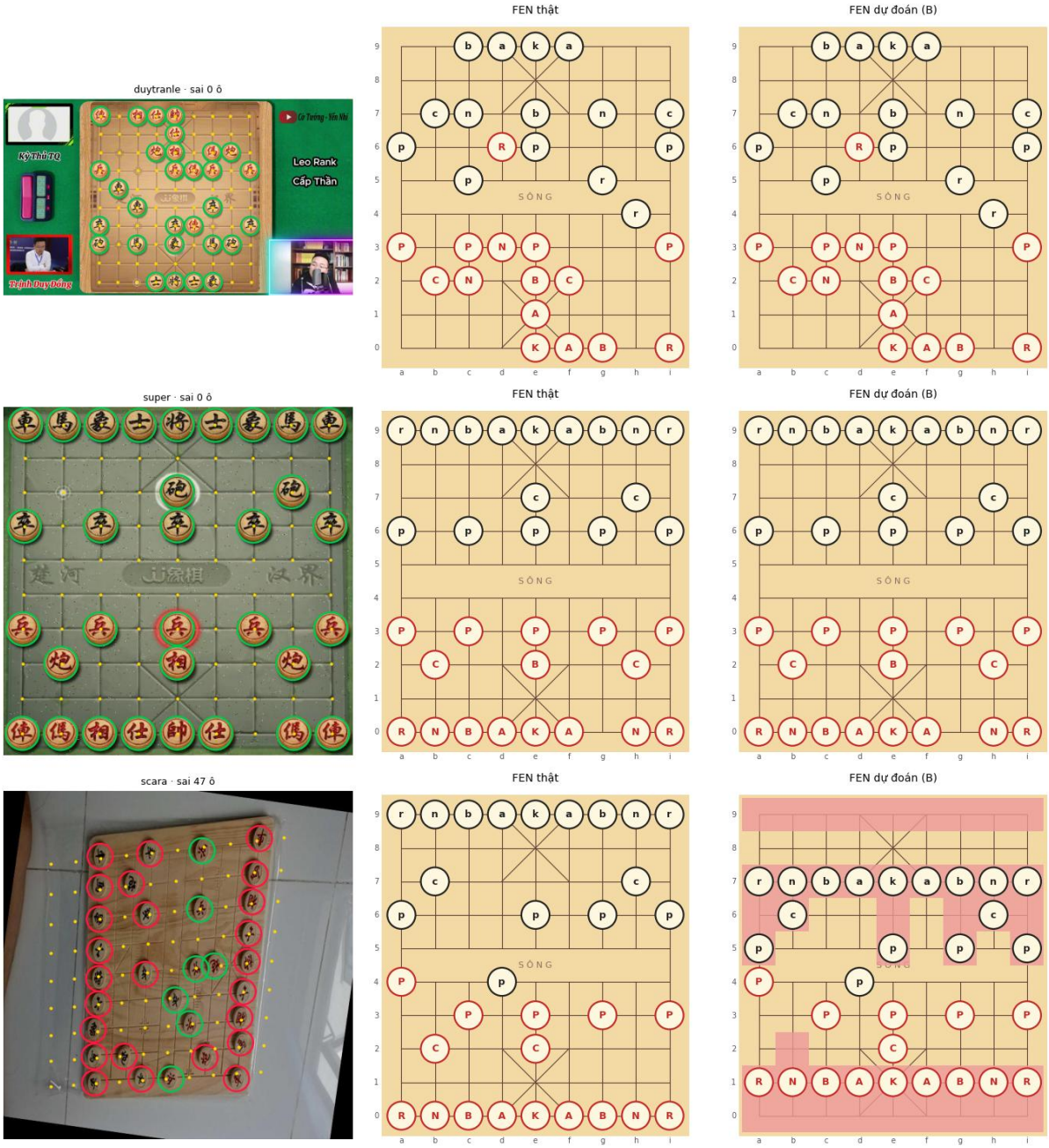
Chỉ số	A · lưới từ nhãn thật	B · end-to-end
Ảnh đúng hoàn toàn	81,7%	78,2%
Sai ≤ 1 ô	96,9%	93,0%
Sai ≤ 2 ô	100,0%	95,6%
Độ chính xác theo quân	99,2%	95,7%

Chỉ số	A · lưới từ nhãn thật	B · end-to-end
Thiếu quân / ảnh	0,05	0,76
Thừa quân / ảnh	0,03	0,64
Sai loại / ảnh	0,14	0,18
Sai màu / ảnh	0,00	0,05



Hình 6. 3. Phân bố số ô sai mỗi ảnh (xanh: đúng hoàn toàn, cam: luật cờ còn sửa được, đỏ: quá nhiều lỗi)

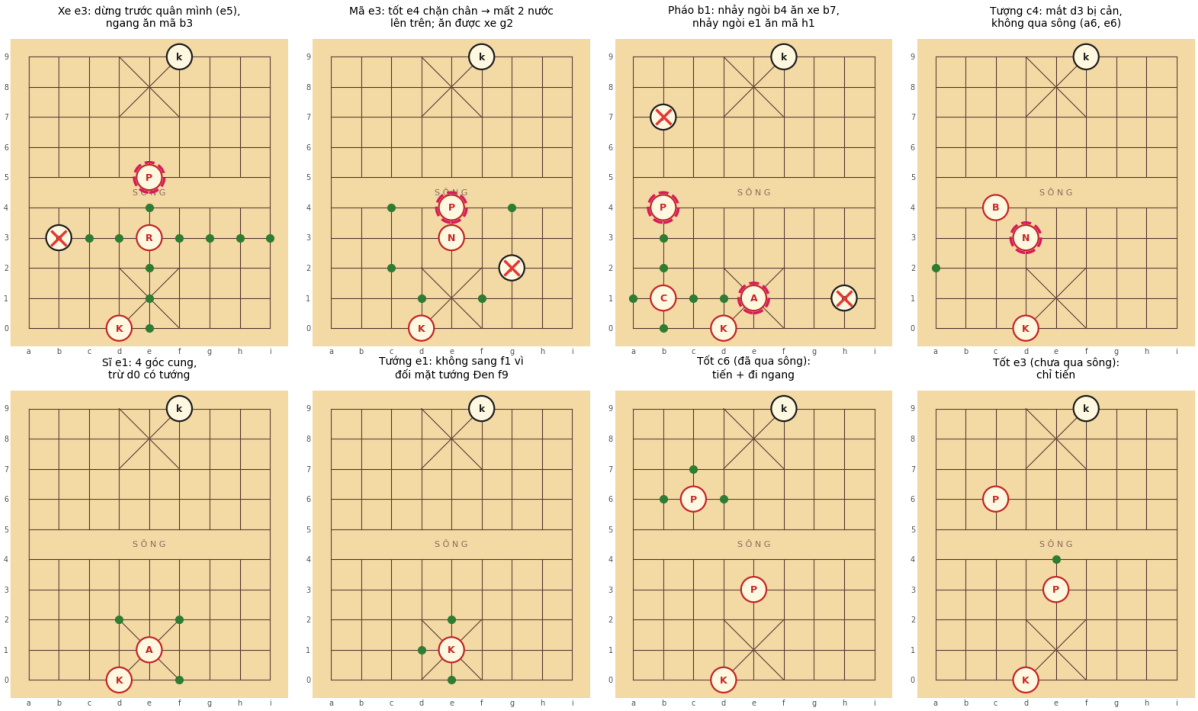
Nhận xét: dù mAP@0.5 đạt 0,964, chỉ khoảng 4/5 số ảnh có **toàn bộ** thế cờ đúng - minh chứng cho việc đánh giá theo từng quân là chưa đủ. Kết quả khác nhau rõ theo nguồn: bộ super (ảnh phần mềm, nhìn thẳng) đúng 95,4%, bộ duytranle (ảnh chụp màn hình livestream, có che khuất) 65,2%, bộ scara (ảnh thật chụp xiên, bàn gỗ có sông rộng) chỉ 6 ảnh vào được tập đánh giá – thuật toán khớp lưới giả định sông rộng đúng 1 ô nên lệch hàng ở vùng sông. Các nhầm lẫn loại quân hay gặp nhất: pháo Đỏ → xe Đỏ (5 lần), mã Đen → xe Đen (3), tượng Đỏ → pháo Đỏ (3).



Hình 6. 4. Ví dụ ảnh tốt nhất, trung vị và tệ nhất: ảnh với lưới khớp được, FEN thật và FEN dự đoán (ô sai tô đỏ)

6.3. Luật cờ và kiểm chứng bộ sinh nước đi

Mô-đun board/rules.py cài đặt luật đi của 7 loại quân (Bảng 2.1). Một nước là **hợp lệ** khi đi đúng cách và sau khi đi, tướng của mình không bị chiếu (kể cả lỗi để hai tướng đối mặt). Việc kiểm tra "bị chiếu" được làm bằng cách nhìn ngược từ ô tướng ra 4 hướng thẳng (xe, pháo, tướng), 8 vị trí mã (có xét chân mã) và các ô tốt – nhanh hơn nhiều so với sinh toàn bộ nước của đối phương.



Hình 6. 5. Nước đi hợp lệ của từng loại quân trong các thế cờ có quân cản (chấm xanh: đi được, ×: ăn được, vòng hồng: quân cản)

Bảng 6. 4. Kiểm chứng bộ sinh nước đi bằng perft từ thế khai cuộc

Độ sâu	board/rules.py	Giá trị chuẩn	Khớp	Thời gian
1	44	44	✓	0,00 s
2	1 920	1 920	✓	0,08 s
3	79 666	79 666	✓	1,80 s
4	3 290 240	3 290 240	✓	53,7 s

Ngoài ra, bộ sinh nước đi được đối chiếu với engine Pikafish (perft độ sâu 2) trên **300 thế cờ giữa trận** sinh ngẫu nhiên (11 thế đang bị chiếu): **không lệch thế nào**.

6.4. Kiểm tra thế cờ nhận dạng được

Hàm validate() phát hiện những thế cờ **chắc chắn không thể xảy ra** trong ván thật - dấu hiệu của lỗi nhận dạng:

Bảng 6. 5. Ví dụ phát hiện thế cờ vô lý

Trường hợp (lỗi nhận dạng mô phỏng)	Lỗi phát hiện
Bỏ sót tướng Đỏ e0	Thiếu tướng Đỏ
Tốt Đen c6 bị nhận nhầm thành pháo	Quá nhiều pháo Đen: 3 > 2
Tượng Đỏ ở c5	Tượng chỉ đứng ở 7 điểm, không qua sông
Tốt Đỏ ở b3	Tốt chưa qua sông chỉ ở cột a, c, e, g, i
Sót tốt e3 và e6	Hai tướng đối mặt trên cột e
Đỏ đi nhưng Đen đang bị chiếu	Bên vừa đi đang bị chiếu

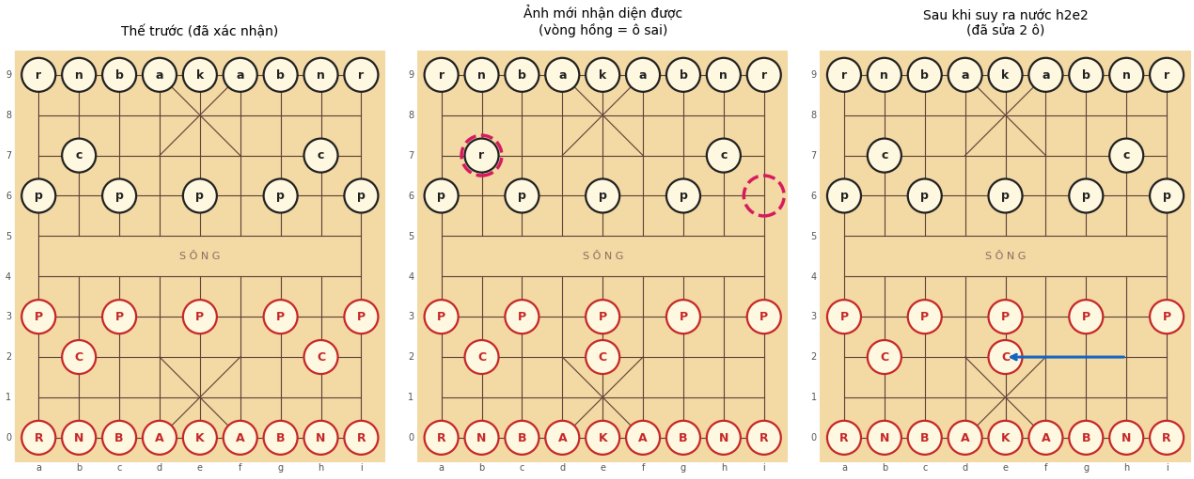
Trên tập đánh giá, luật cờ phát hiện được 48,0% số FEN sai (thiết lập B) và **không báo nhầm** FEN đúng nào. Phần còn lại là lỗi "hợp lý" (ví dụ sót một quân tốt) – chỉ phát hiện được khi biết thế cờ lượt trước.

6.5. Suy ra nước vừa đi và tự sửa lỗi nhận dạng

Khi đã biết thế cờ lượt trước S (đã xác nhận đúng), bên đến lượt chỉ có khoảng 40 nước hợp lệ. Thay vì tin thẳng ảnh mới Ŝ, thư viện chọn nước đi m *mà thế cờ sau nước đó giống ảnh mới nhất**:

$$m^* = \operatorname{argmin} \{ d(S \oplus m, \hat{S}) : m \in \text{Hợp_lệ}(S) \cup \{\emptyset\} \}, \quad d = \text{số ô khác nhau trên 90 giao điểm}$$

Nếu d(m*) = 0: ảnh đúng hoàn toàn; nếu 0 < d(m)* ≤ 2: **đã tự sửa** các ô sai; nếu lớn hơn hoặc có hai nước cùng khoảng cách: **không chắc chắn → chụp lại**. Một nước đi chỉ thay đổi 2 ô, nên hai nước hợp lệ khác nhau cho hai thế cờ khác nhau ít nhất 2 ô - sai 1 ô vẫn luôn chọn đúng nước.



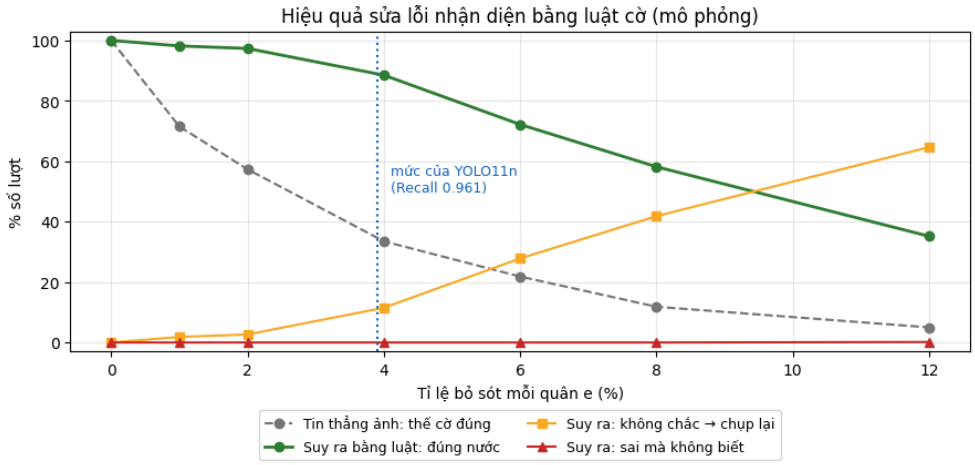
Hình 6. 6. Ví dụ: ảnh mới có 2 ô nhận sai (pháo b7 nhận thành xe, sót tốt i6) – thư viện vẫn suy ra đúng nước "Pháo 2 bình 5" và sửa cả 2 ô

6.5.1. Thực nghiệm mô phỏng

Trên 600 thế cờ từ ván ngẫu nhiên (trung bình 21 quân), lỗi nhận dạng được mô phỏng với tỉ lệ bỏ sót e mỗi quân, nhầm loại e/3 mỗi quân và báo nhầm e mỗi ảnh:

Bảng 6. 6. Hiệu quả sửa lỗi bằng luật cờ (mô phỏng)

Tỉ lệ lỗi e	Tin thẳng ảnh: đúng	Suy ra: đúng nước	Chụp lại	Sai mà không biết
0%	100,0%	100,0%	0,0%	0,0%
1%	71,5%	98,2%	1,8%	0,0%
2%	57,3%	97,3%	2,7%	0,0%
4% (≈ YOLO11n)	33,5%	88,5%	11,5%	0,0%
6%	21,8%	72,2%	27,8%	0,0%
8%	11,8%	58,2%	41,8%	0,0%
12%	5,0%	35,2%	64,7%	0,2%



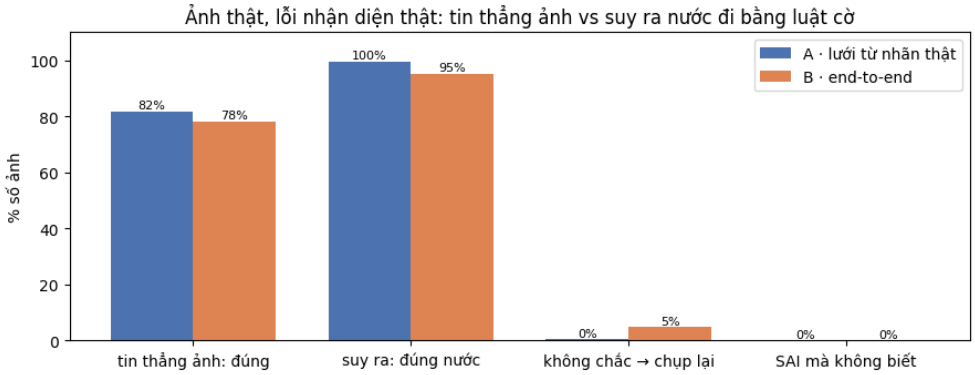
Hình 6. 7. Hiệu quả sửa lỗi nhận diện bằng luật cờ theo tỉ lệ lỗi (mô phỏng)

6.5.2. Thực nghiệm với lỗi nhận dạng thật

Trên 229 ảnh kiểm tra, lấy FEN thật làm thế lượt trước, chọn ngẫu nhiên một nước hợp lệ, rồi dùng **FEN dự đoán của chính ảnh đó** (lỗi thật của mô hình) với quân ở ô đi được chuyển sang ô đến làm "ảnh mới":

Bảng 6. 7. Tin thẳng ảnh và suy ra nước đi bằng luật cờ trên lỗi nhận dạng thật

Kết quả	A · lưới từ nhãn thật	B · end-to-end
Tin thẳng ảnh: đúng	81,7%	78,2%
Suy ra bằng luật: đúng nước	99,6%	95,2%
Không chắc → chụp lại	0,4%	4,8%
Sai mà không biết	0,0%	0,0%



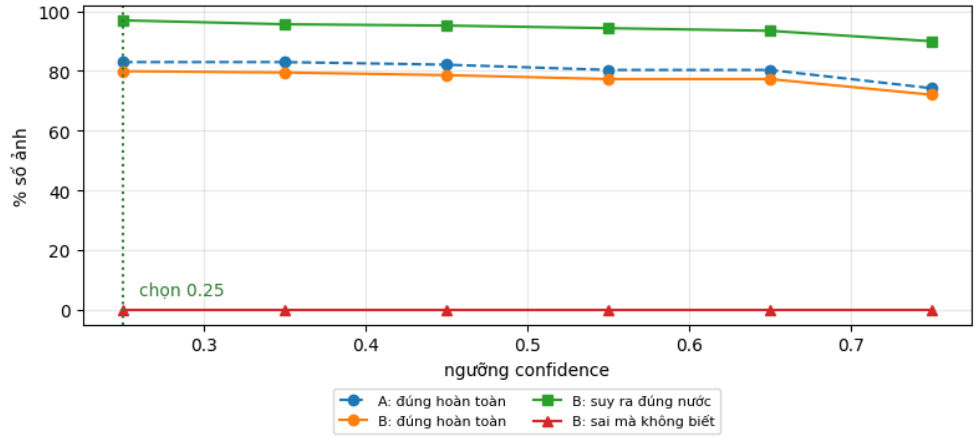
Hình 6. 8. Ảnh thật, lỗi nhận diện thật: tin thẳng ảnh và suy ra nước đi bằng luật cờ

6.6. Chọn ngưỡng tin cậy

Với bài toán thế cờ, chỉ số cần tối ưu là **tỉ lệ suy ra đúng nước**, không phải mAP. Quét ngưỡng tin cậy từ 0,25 đến 0,75:

Bảng 6. 8. Ảnh hưởng của ngưỡng tin cậy

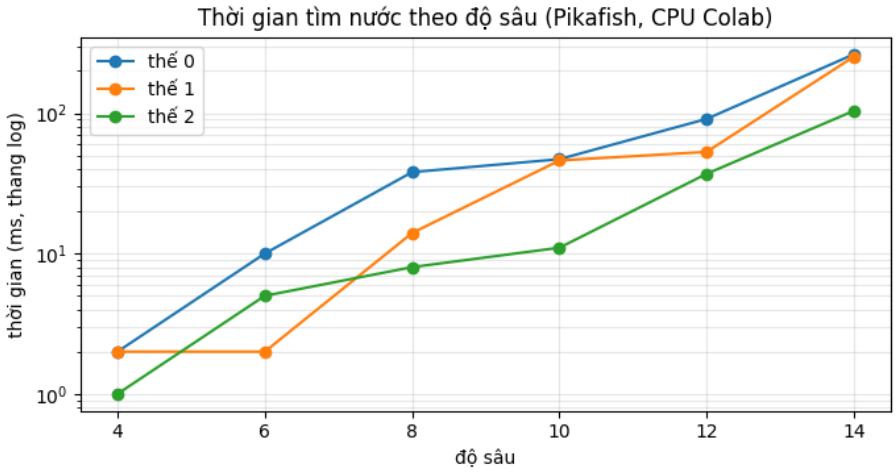
Ngưỡng	A: đúng hoàn toàn	B: đúng hoàn toàn	B: suy ra đúng nước	B: sai mà không biết
0,25	83,0%	79,9%	96,9%	0,0%
0,35	83,0%	79,5%	95,6%	0,0%
0,45	82,1%	78,6%	95,2%	0,0%
0,55	80,3%	77,3%	94,3%	0,0%
0,65	80,3%	77,3%	93,4%	0,0%
0,75	74,2%	72,1%	90,0%	0,0%



Hình 6. 9. Tỉ lệ đúng theo ngưỡng tin cậy; ngưỡng 0,25 được chọn làm mặc định của thư viện

6.7. Engine Pikafish

Pikafish được biên dịch từ mã nguồn (phiên bản dev-20261029, mạng NNUE 48 MB) và điều khiển qua lớp UciEngine. Thời gian tìm nước tăng gần theo hàm mũ với độ sâu; trên CPU Colab, độ sâu 14 mất khoảng 0,1 - 0,26 giây. Jetson Nano chậm hơn nhiều lần và phải chia CPU với camera, nên độ sâu mặc định của thư viện là 8.



Hình 6. 10. Thời gian tìm nước của Pikafish theo độ sâu trên 3 thế cờ (CPU Colab)

Phép thử tích hợp: cho Pikafish tự đánh với chính nó ở độ sâu 10 - cả **51 nước** đều được board/rules.py xác nhận hợp lệ, ván dừng khi một thế cờ lặp lại 3 lần.

CHƯƠNG 7. XÂY DỰNG THƯ VIỆN, GIAO DIỆN ỨNG DỤNG VÀ TRIỂN KHAI

7.1. Thư viện XiangqiBrain

Lớp XiangqiBrain (brain/xiangqi_brain.py) gom toàn bộ các bước trên thành hai cách sử dụng:

Bảng 7. 1. Các hàm chính của lớp XiangqiBrain

Hàm	Đầu vào	Đầu ra
analyze(img, prev_fen=None, side=None)	Ảnh BGR; FEN lượt trước (tuỳ chọn); bên đi	status, fen (đã sửa lỗi), last_move, best_move, best_move_from / to, captures, score_cp
step(img)	Ảnh sau khi người đi	status 'robot_move' / 'waiting' / 'retake' / 'game_over'; human_move; robot_move
confirm(move)	Nước robot vừa đi xong	Cập nhật ván cờ
read_board(img)	Ảnh BGR	Thế cờ thô (chưa dùng luật), danh sách quân, cách định vị bàn

```
from brain.xiangqi_brain import XiangqiBrain
brain = XiangqiBrain()                          # đọc cấu hình config/robot.yaml
res = brain.analyze(cv2.imread('anh_moi.jpg'), prev_fen=fen_luot_truoc)
if res['status'] == 'ok':
    print(res['last_move_vn'])                  # 'Pháo 2 bình 5'  (đối thủ vừa đi)
    print(res['best_move_from'], res['best_move_to'])   # 'b9' 'c7'        (ô gắp -> ô đặt)
    fen_luot_truoc = res['fen']                 # lưu cho lượt sau
```

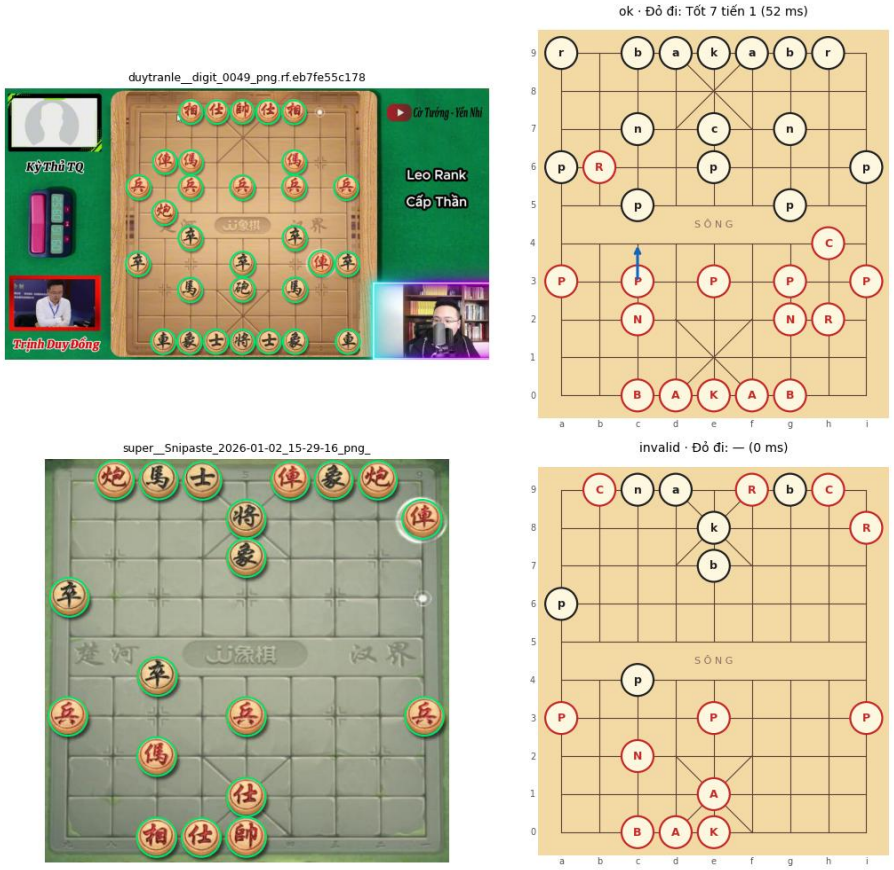
Các giá trị mặc định của cấu hình được lấy từ kết quả thực nghiệm: ngưỡng tin cậy 0,25 (mục 6.6), chạy YOLO một lần không xoay ảnh (mục 5.6), tự sửa tối đa 2 ô (mục 6.5); định vị bàn thử board_detect trước, không thấy bàn thì dùng lattice. Thư viện có tài liệu hướng dẫn gọi API đầy đủ (API.md).

7.2. Kiểm thử tự động

Hai bộ kiểm thử chạy được cả trên Colab, laptop và robot:

- tests/test_rules.py: đọc/ghi FEN, perft 1–3, phát hiện thế cờ vô lý, ký hiệu tiếng Việt, suy ra nước và sửa 2 ô sai, khớp lưới trên thế khai cuộc xoay 23°, engine dự phòng và Pikafish.
- tests/test_brain.py: chạy thư viện trên **ảnh bàn cờ thật** của đề tài với bộ nhận diện giả đặt quân đúng giao điểm – kiểm tra định vị bàn (cả hai cách), sửa lỗi, gợi ý nước, phát hiện thế cờ vô lý và chơi nhiều lượt liên tiếp.

Kết quả trên Colab: tất cả các mục đều đạt (OK). Ngoài ra, thư viện thật (YOLO bằng ONNX Runtime + Pikafish) được chạy lại trên 120 ảnh kiểm tra lấy ngẫu nhiên: **83,3%** ảnh ra FEN đúng hoàn toàn, 98,3% sai ≤ 2 ô, thời gian ảnh → thế cờ trung vị 350 ms trên CPU Colab; theo từng nguồn kết quả bằng hoặc cao hơn Bước 4 (super 96,6% so với 95,4%; duytranle 72,9% so với 65,2%) nhờ ngưỡng tin cậy 0,25.

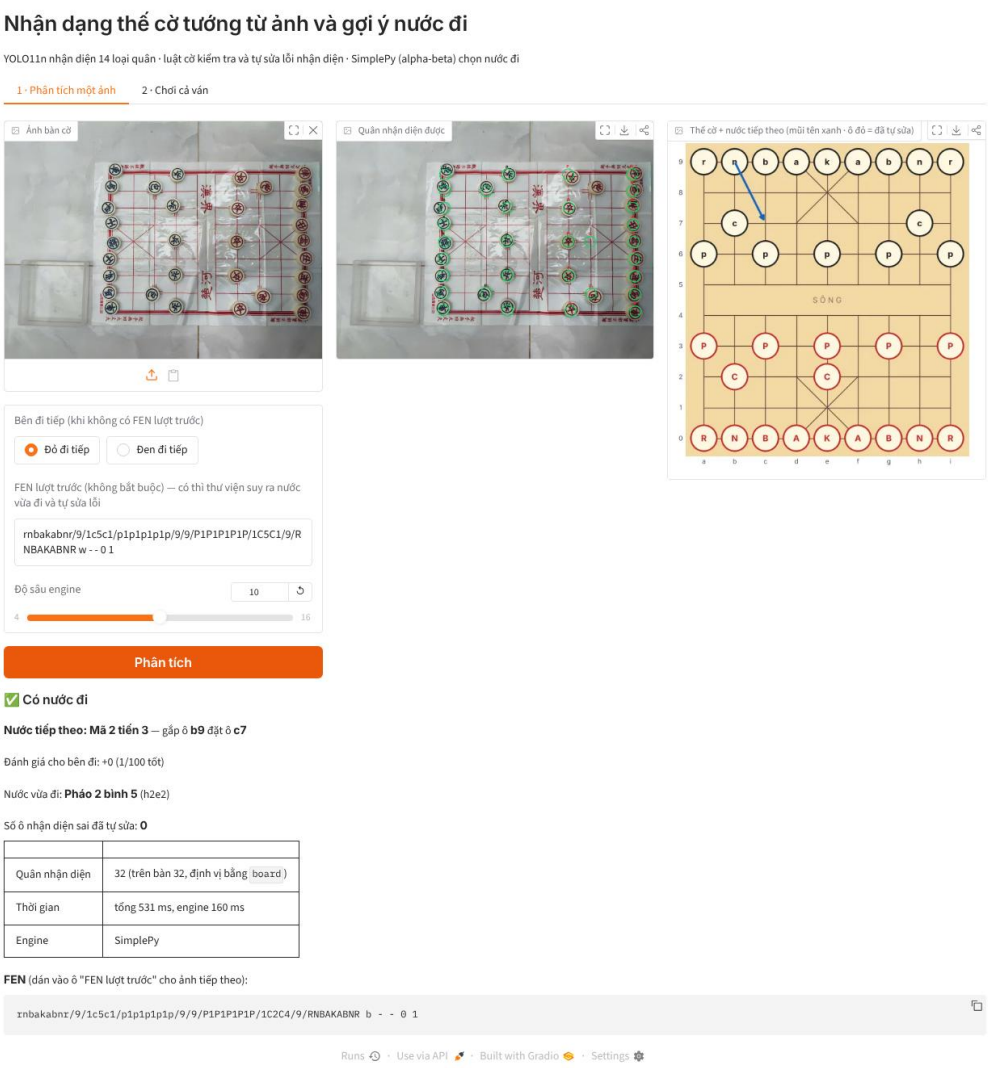


Hình 7. 1. Thư viện chạy trọn chuỗi ảnh → nước đi trên ảnh kiểm tra (trên) và phát hiện một thế cờ vô lý (dưới)

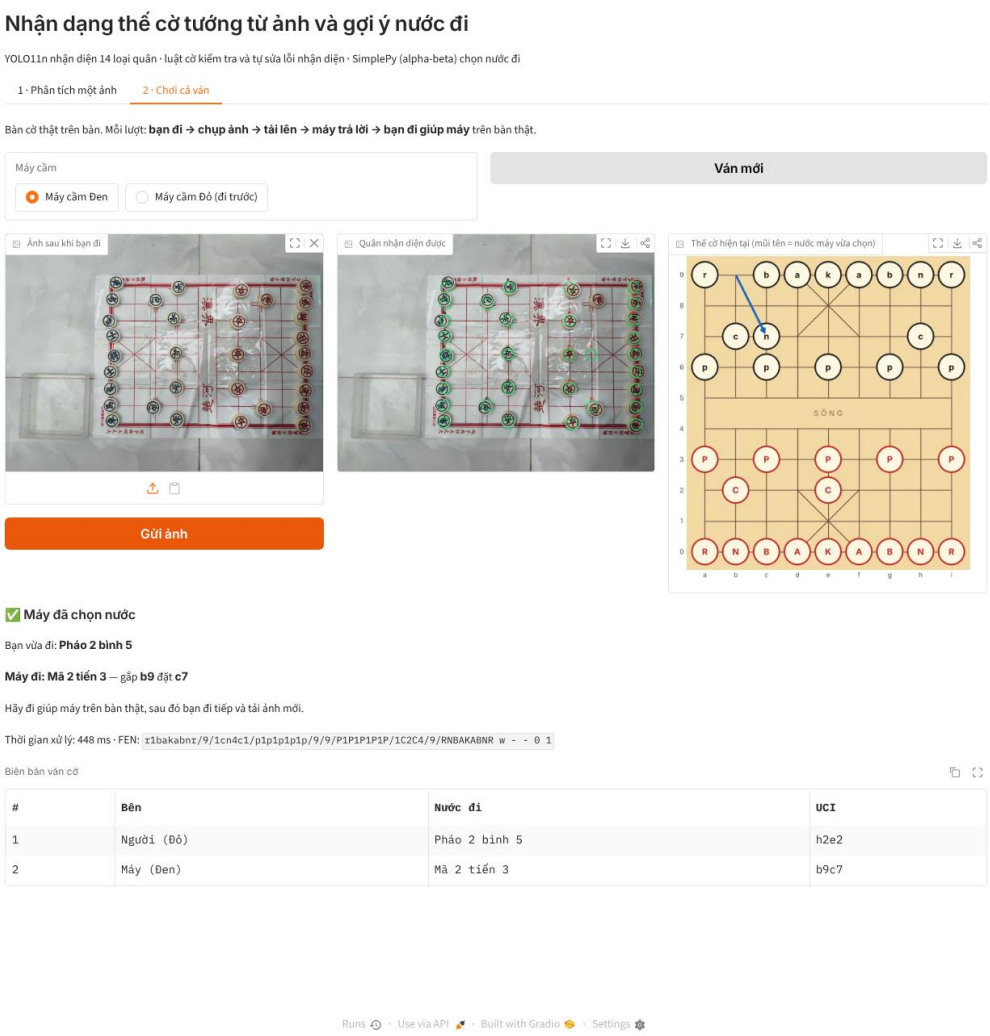
7.3. Giao diện ứng dụng

Giao diện ứng dụng được xây dựng bằng thư viện **Gradio**, chạy trong trình duyệt trên laptop (Windows / Linux) hoặc Colab, không cần webcam: người dùng **tải ảnh bàn cờ lên** và nhận kết quả. Giao diện gồm hai thẻ:

- **Phân tích một ảnh:** tải ảnh lên, chọn bên đi hoặc dán FEN lượt trước; hiển thị ảnh với các quân nhận diện được khoanh tròn, bàn cờ vẽ lại theo FEN với mũi tên nước gợi ý, **các ô đã tự sửa được tô đỏ**, cùng FEN, ký hiệu tiếng Việt và thời gian xử lý.
- **Chơi cả ván:** mỗi lượt người chơi đi quân trên bàn thật, chụp và tải ảnh lên; máy trả lời nước đi (ô gắp → ô đặt) và ghi biên bản ván cờ bằng ký hiệu tiếng Việt.



Hình 7. 2. Giao diện thẻ "Phân tích một ảnh": ảnh tải lên, quân nhận diện được, thế cờ và nước gợi ý

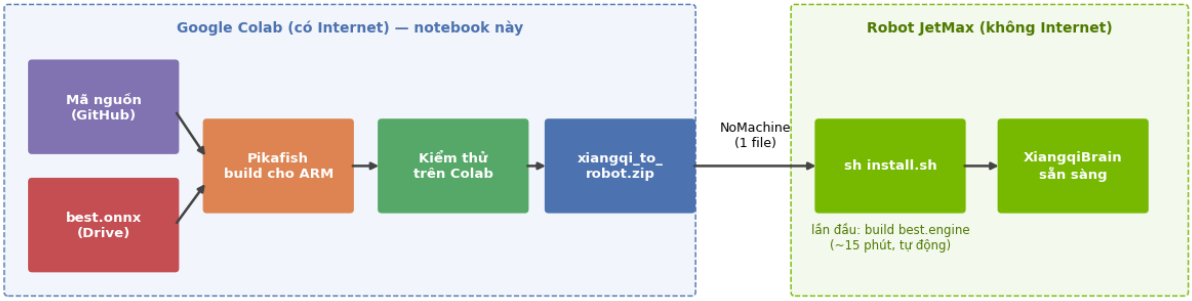


Hình 7. 3. Giao diện thẻ "Chơi cả ván": máy trả lời nước đi và ghi biên bản ván cờ

Trên laptop Windows, người dùng chỉ cần cài Python, giải nén gói xiangqi_demo_windows.zip (mã nguồn, mô hình ONNX, Pikafish bản Windows) và chạy tệp run_demo.bat; trình duyệt tự mở địa chỉ http://127.0.0.1:7860.

7.4. Đóng gói và triển khai lên robot JetMax

Robot không có Internet và chỉ có Python 3.6, nên mọi thứ được chuẩn bị sẵn trên Colab và mang sang trong **một tệp** xiangqi_to_robot.zip (61,2 MB): mã nguồn, best.onnx, Pikafish biên dịch chéo cho ARM (aarch64, liên kết tĩnh) và tệp cài đặt install.sh. Trên robot chỉ cần chạy một lệnh; install.sh tự build best.engine (TensorRT FP16, khoảng 15 phút ở lần đầu), kiểm tra Pikafish và chạy bộ kiểm thử, đồng thời giữ nguyên các số đo hiệu chỉnh riêng của robot.



Hình 7. 4. Quy trình đóng gói trên Colab và cài đặt trên robot JetMax

```
cd ~ && python3 -m zipfile -e ~/Desktop/xiangqi_to_robot.zip ~/
cd ~/xiangqi && sh install.sh
```

Pikafish không được build trên robot vì cần trình biên dịch C++17 mới hơn GCC 7 của Ubuntu 18.04; trong khi mô hình YOLO phải build trên robot vì engine TensorRT gắn với đúng GPU. Bản Pikafish cho ARM đã được kiểm tra chạy được qua giả lập.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết quả đạt được

- Xây dựng bộ dữ liệu 6 808 ảnh / 112 938 quân / 14 lớp từ 4 nguồn công khai; làm sạch nhãn, chuyển polygon/pose về hộp bao, gom 4 265 nhóm ảnh trùng bằng pHash và chia tập không rò rỉ.
- Trình bày và cài đặt đầy đủ chuỗi **tiền xử lý ảnh**: letterbox, đổi không gian màu, chuẩn hoá, tăng cường dữ liệu có chọn lọc (xoay ±180°, không lật), định vị bàn cờ bằng HSV – hình thái học – Hough – homography, xác định màu quân bằng phân ngưỡng HSV, và mô-đun **tăng cường ảnh tự động** (lọc song phương, Gray-World, CLAHE, gamma) giúp tìm lại bàn cờ trên ảnh nhiễu mạnh.
- Huấn luyện YOLO11n bằng thư viện Ultralytics: **mAP@0.5 = 0,964**, Precision 0,990, Recall 0,959 trên 713 ảnh kiểm tra; không suy giảm khi ảnh xoay (0,963 - 0,965).
- Thuật toán khớp lưới từ tâm quân, khôi phục đúng 96 - 98% thế cờ ≥ 20 quân trên dữ liệu tổng hợp.
- Dùng luật cờ để tự sửa lỗi: ảnh → FEN đúng 78,2%, khi biết thế trước suy ra đúng nước **95,2%** (ngưỡng 0,25: 96,9%) và **0% sai mà không phát hiện**.
- Thư viện XiangqiBrain có tài liệu API, bộ kiểm thử tự động, giao diện ứng dụng web và gói cài đặt chạy offline cho robot JetMax.

Hạn chế

- Đánh giá thực hiện trên dữ liệu công khai, chưa đo trên camera của robot JetMax.
- Thuật toán khớp lưới giả định sông rộng bằng 1 ô; bàn cờ chụp xiên mạnh hoặc thế cờ thưa (< 15 quân) có thể khớp sai (bộ scara).
- Tăng cường ảnh mới được đánh giá trên ảnh suy giảm mô phỏng và chỉ áp dụng cho bước định vị bàn cờ; chưa đo ảnh hưởng tới mô hình YOLO11n trên ảnh thật từ camera robot.
- Lớp black_cannon và red_advisor còn tỉ lệ bỏ sót cao hơn các lớp khác.

Hướng phát triển

- Chụp bộ ảnh từ camera robot (thế cờ đã biết) để đo độ chính xác thật và tinh chỉnh mô hình.
- Thu ảnh xấu thật từ camera robot (thiếu sáng, đèn vàng, nhiễu) để đo ảnh hưởng của tăng cường ảnh lên YOLO11n và quyết định bật cho nhánh nhận diện quân (apply_to: pieces).
- Mô hình hoá độ rộng sông và độ cao quân trong thuật toán khớp lưới.
- Ghép với điều khiển tay máy (hiệu chỉnh camera – robot, gắp – đặt) để robot tự đi quân.
- Dùng độ tin cậy của mô hình làm trọng số khi sửa lỗi bằng luật cờ.

TÀI LIỆU THAM KHẢO

[1] R. C. Gonzalez, R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.

[2] R. Hartley, A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004.

[3] G. Bradski, "The OpenCV Library", *Dr. Dobb's Journal of Software Tools*, 2000. https://docs.opencv.org/4.x/

[4] R. O. Duda, P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures", *Communications of the ACM*, 15(1), 1972.

[5] J. Matas, C. Galambos, J. Kittler, "Robust detection of lines using the progressive probabilistic Hough transform", *CVIU*, 78(1), 2000.

[6] J. Canny, "A computational approach to edge detection", *IEEE TPAMI*, 8(6), 1986.

[7] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization", *Graphics Gems IV*, Academic Press, 1994.

[8] C. Tomasi, R. Manduchi, "Bilateral filtering for gray and color images", *ICCV*, 1998.

[9] C. Zauner, *Implementation and Benchmarking of Perceptual Image Hash Functions*, Master thesis, Upper Austria University of Applied Sciences, 2010.

[10] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection", *CVPR*, 2016.

[11] G. Jocher, J. Qiu, *Ultralytics YOLO11*, 2024. https://github.com/ultralytics/ultralytics

[12] Ultralytics, *Ultralytics YOLO Docs* (Train, Val, Predict, Export, Configuration). https://docs.ultralytics.com/vi

[13] S. Ren, K. He, R. Girshick, J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", *NeurIPS*, 2015.

[14] S. Liu, L. Qi, H. Qin, J. Shi, J. Jia, "Path Aggregation Network for Instance Segmentation", *CVPR*, 2018.

[15] Z. Zheng et al., "Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression", *AAAI*, 2020.

[16] X. Li et al., "Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection", *NeurIPS*, 2020.

[17] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context", *ECCV*, 2014.

[18] ONNX, *Open Neural Network Exchange*. https://onnx.ai/ ; ONNX Runtime. https://onnxruntime.ai/

[19] NVIDIA, *TensorRT Developer Guide*. https://docs.nvidia.com/deeplearning/tensorrt/

[20] Pikafish developers, *Pikafish – a free and strong UCI xiangqi engine*. https://github.com/official-pikafish/Pikafish

[21] Y. Nasu, *Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi*, 2018.

[22] D. E. Knuth, R. W. Moore, "An analysis of alpha-beta pruning", *Artificial Intelligence*, 6(4), 1975.

[23] Chessprogramming Wiki, *Chinese Chess Perft Results*. https://chessprogramming.org/Chinese_Chess_Perft_Results

[24] scikit-learn, *StratifiedGroupKFold*. https://scikit-learn.org/

[25] Roboflow Universe – các bộ dữ liệu: xiangqi-super-dataset, xiangqi-scara-yolo26 (312s-workspace), new chinese chess (quoc-rnmzr), xiangqi (duytranle). https://universe.roboflow.com/

[26] Gradio, *Build & Share Delightful Machine Learning Apps*. https://www.gradio.app/

[27] Hiwonder, *JetMax – JETSON NANO Robot Arm*; NVIDIA, *Jetson Nano Developer Kit*, *JetPack SDK 4.6.1*.

PHỤ LỤC

Phụ lục A. Cấu trúc mã nguồn

Đường dẫn	Nội dung
training/build_dataset.ipynb	Bước 1: xây dựng bộ dữ liệu hợp nhất
training/train_yolo11.ipynb	Bước 2: huấn luyện, đánh giá, xuất ONNX
training/jetmax_deploy.ipynb	Bước 3: robot JetMax, đưa mô hình và engine lên robot
training/board_to_fen.ipynb	Bước 4: đánh giá ảnh → FEN trên tập kiểm tra
training/rules_engine.ipynb	Bước 5: luật cờ, kiểm tra thế cờ, sửa lỗi, Pikafish
training/package.ipynb	Bước 6: kiểm thử thư viện và đóng gói
vision/pieces.py, board_detect.py, lattice.py	Nhận diện quân, định vị bàn cờ, khớp lưới
board/rules.py, geometry.py, draw.py	Luật cờ, hình học bàn cờ, vẽ bàn cờ
brain/xiangqi_brain.py, game.py, uci.py, simple.py	Thư viện chính, theo dõi ván, Pikafish, engine dự phòng
tools/demo_app.py, run_demo.bat	Giao diện ứng dụng
tests/test_rules.py, test_brain.py	Kiểm thử tự động
API.md, DEMO_WINDOWS.md, RUNBOOK.md	Tài liệu API, hướng dẫn demo, hướng dẫn robot

Phụ lục B. Hướng dẫn chạy giao diện ứng dụng trên Windows

1) Cài Python 3.11 (tích chọn "Add python.exe to PATH").
2) Giải nén xiangqi_demo_windows.zip vào thư mục không dấu, ví dụ D:\xiangqi_demo.
3) Bấm đúp run_demo.bat: lần đầu tự cài thư viện (cần Internet, 3 – 5 phút), sau đó trình duyệt tự mở http://127.0.0.1:7860.
4) Chụp ảnh bàn cờ thẳng từ trên xuống, phía quân Đỏ ở cạnh dưới ảnh, rồi tải lên giao diện.